



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации
информационных технологий

Отчет по практической работе №6

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 BottomBar	3
1.2 ActionBar	11
1.3 Navigation Drawer	16
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	23
2.1 Создание интерактивного каталога в всплывающем меню.....	23
2.2 Реализация интеграции BottomBar с ActionBar	29
ЗАКЛЮЧЕНИЕ	34

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 BottomBar

В предыдущих практических работах были рассмотрены переходы между разными экранами в приложении с использованием простых элементов Button. Однако, в контексте современной Android разработки такой метод является не интуитивным и устаревшим. Поэтому, на замену такой навигации были придуманы объекты пользовательского интерфейса BottomBar и Drawer.

BottomBar, или нижняя панель навигации, является ключевым элементом пользовательского интерфейса в мобильных приложениях, работающих под управлением Android. Этот элемент предназначен для улучшения навигации и повышения удобства использования приложения за счёт предоставления быстрого доступа к основным разделам приложения. В контексте перемещения между экранами BottomBar выполняет несколько важных функций:

- улучшение пользовательского опыта: BottomBar делает навигацию по приложению интуитивно понятной и удобной. Располагаясь в нижней части экрана, он легко доступен для пользователя, что особенно важно при использовании устройства одной рукой. Это облегчает переход между ключевыми разделами приложения, не отвлекая пользователя от основного контента;
- повышение производительности: использование BottomBar позволяет сократить количество нажатий, необходимых для перехода между разделами приложения, тем самым ускоряя взаимодействие пользователя с приложением и повышая его общую производительность;
- организация контента: BottomBar помогает организовать контент в приложении, выделяя основные разделы или функции, которые должны

быть всегда под рукой. Это упрощает структуру приложения и делает его более понятным для пользователя.

BottomBar представляет собой контейнер, который содержит в себе различные элементы (рисунок 1.1.1).

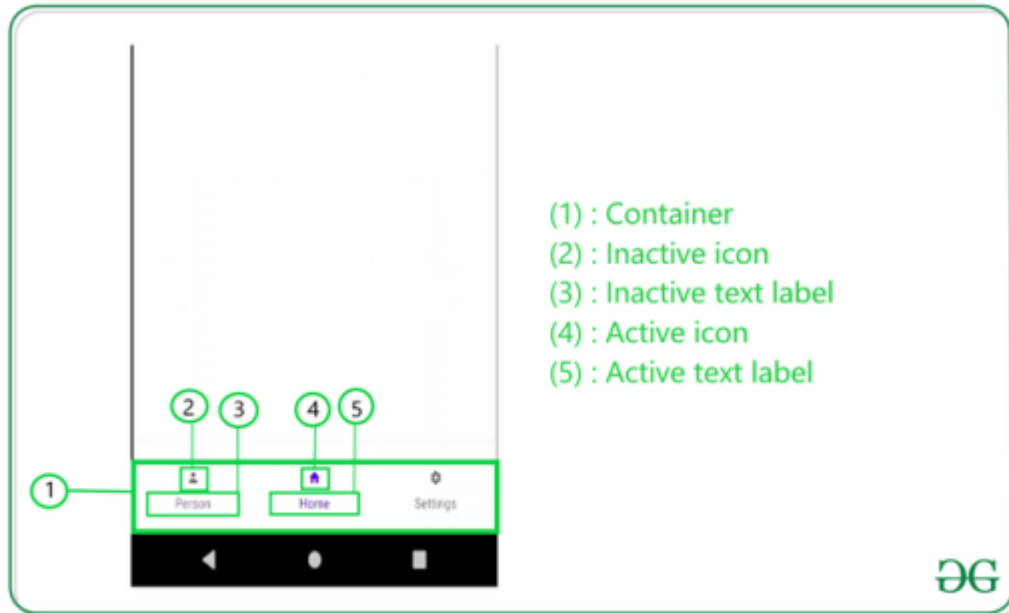


Рисунок 1.1.1 – Наполнение BottomBar-a

Для использования BottomNavigationView необходимо добавить этот элемент в макет активности (рисунок 1.1.2).

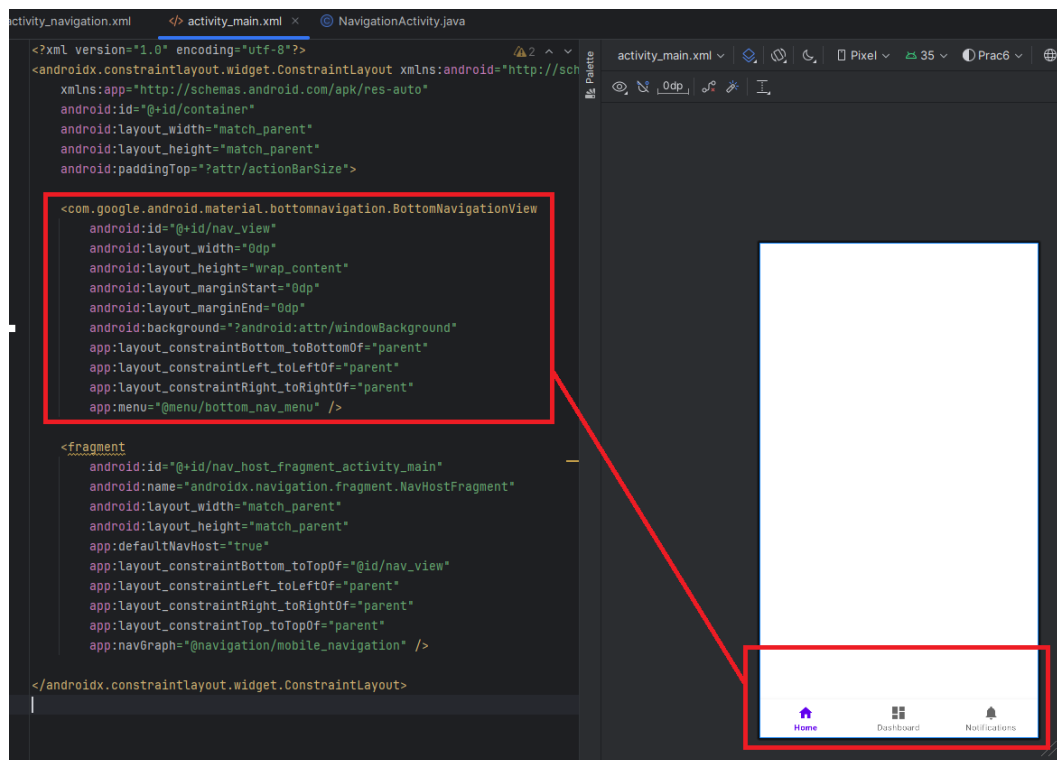


Рисунок 1.1.2 – Описание BottomNavigationView в файле разметки

CoordinatorLayout – это контейнер, который расширяет FrameLayout.

CoordinatorLayout предназначен для двух основных вариантов использования:

- в качестве декора приложения верхнего уровня или макета Chrome;
- как контейнер для определенного взаимодействия с одним или несколькими дочерними представлениями.

После добавления BottomNavigationView в активность необходимо заполнить его элементами (экранами, между которыми будет осуществляться перемещение). Для этого создается XML-файл в папке res/menu (например, bottom_nav_menu.xml) и добавляются в него пункты меню. Если папка menu отсутствует в проекте, то сначала ее необходимо добавить.

Для создания новой папки нажимаем правой кнопкой мыши на "res" → "New" → "Android Resource Directory" (рисунок 1.1.3). Вводим название папки, выбираем тип ресурса из выпадающего списка (рисунок 1.1.4).

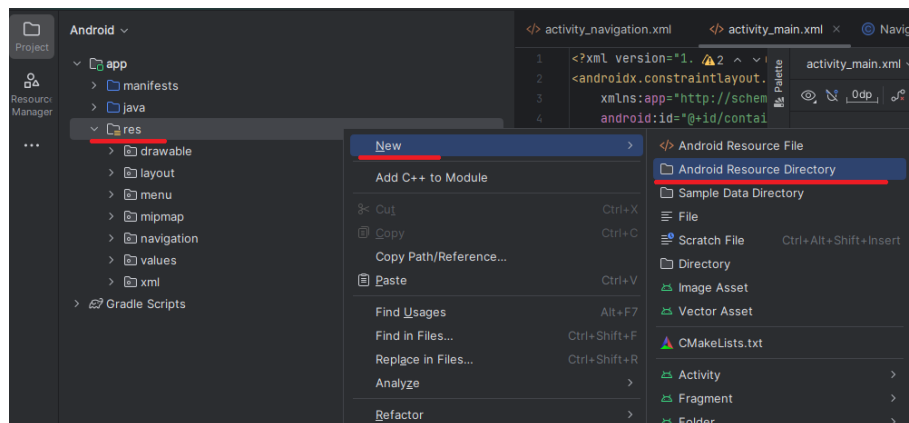


Рисунок 1.1.3 – Создание новой папки в проекте

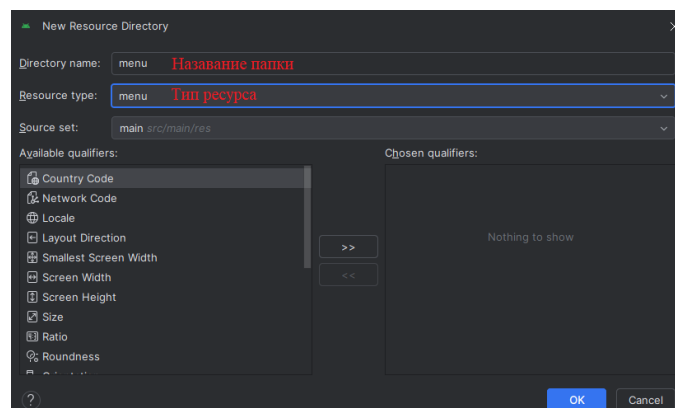


Рисунок 1.1.4 – Настройка создаваемой папки

После создания XML-файла его можно наполнить пунктами меню

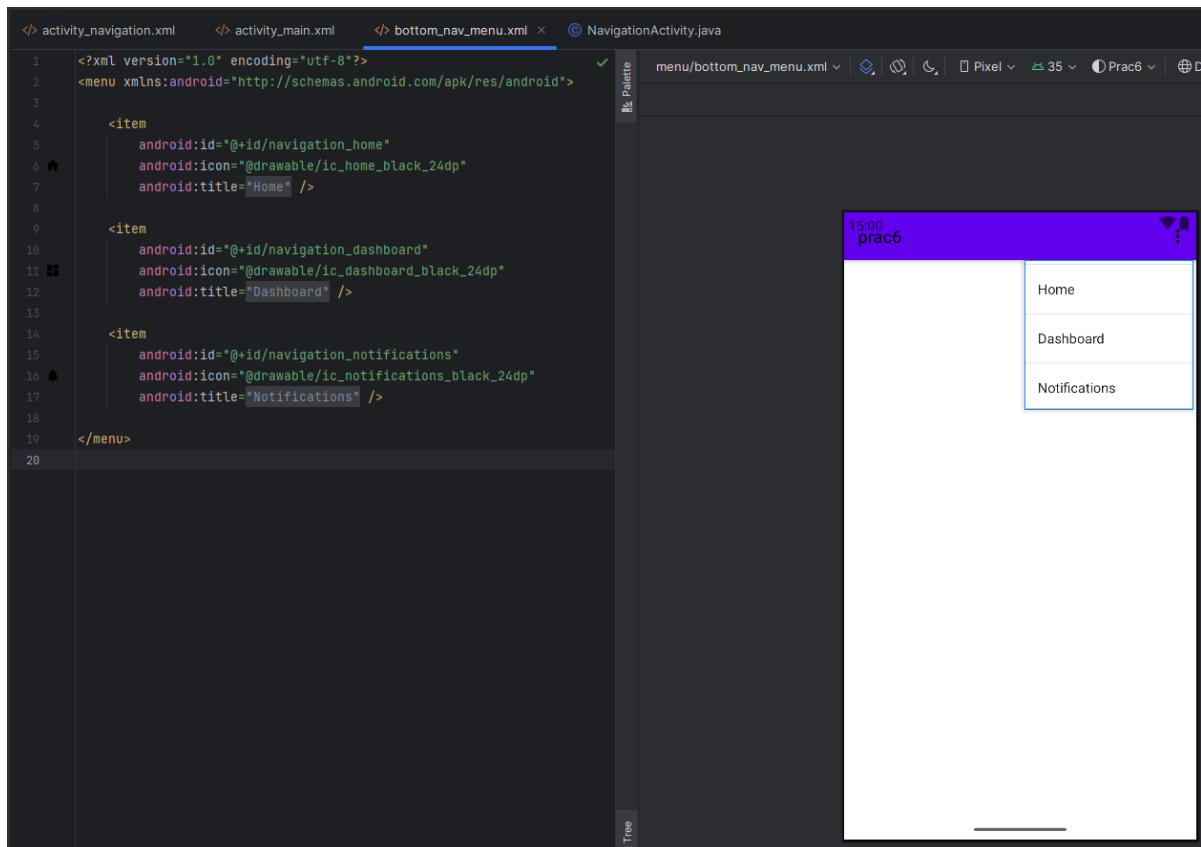


Рисунок 1.1.5 - Графический и кодовый вид нижнего меню

Тег `<menu>` является корневым узлом файла и определяет меню, состоящее из одного или нескольких элементов `<item>` и `<group>`.

Элемент `<item>` представляет объект `MenuItem`, которой является одним из элементов меню. Этот элемент может содержать внутренний подэлемент `<menu>`, с помощью которого создается подменю.

Элемент `<item>` включает следующие атрибуты, которые определяют его внешний вид и поведение:

- `android:id`: уникальный id элемента меню, который позволяет его опознать при выборе пользователем и найти через поиск ресурса по id;
- `android:icon`: ссылка на ресурс drawable, который задает изображение для элемента (`android:icon="@drawable/ic_help"`);
- `android:title`: ссылка на ресурс строки, содержащий заголовок элемента.

По умолчанию имеет значение "Settings";

Теперь пользователь может создавать столько элементов, сколько захочет, в `bottom_nav_menu.xml` файле. Пользователю также необходимо

создать значок для отображения каждого из этих элементов. Чтобы создать значок, нажмите на "drawable" → "New" → "Image Asset" (рисунок 1.1.6).

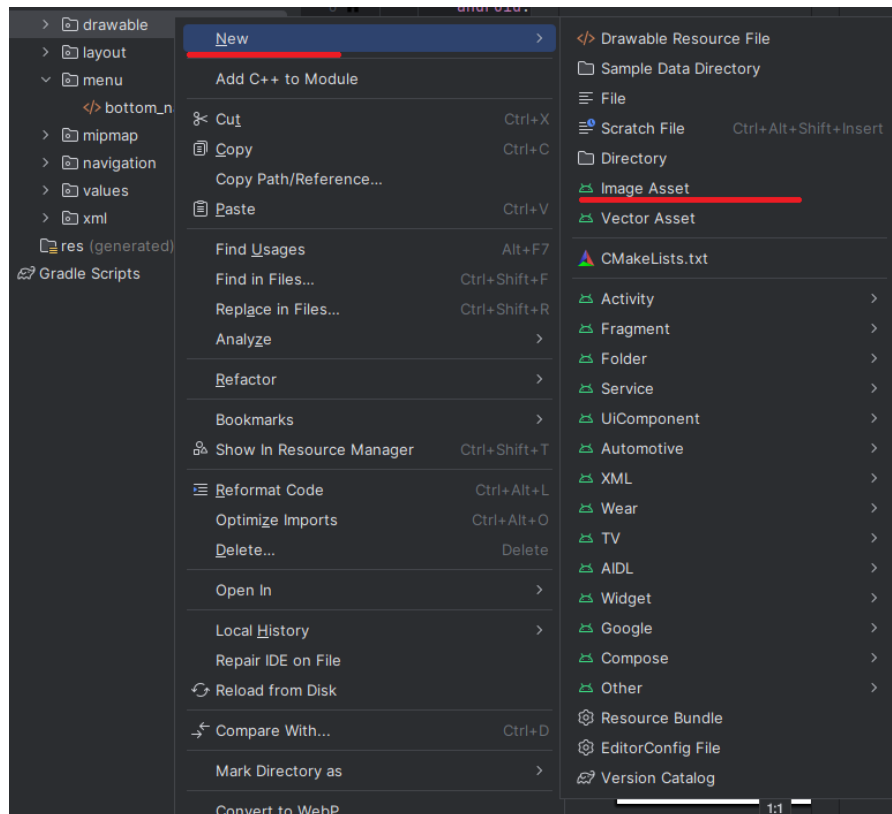


Рисунок 1.1.6 – Процесс создания новой иконки

Далее идёт выбор нужного типа иконки (рисунок 1.1.7).

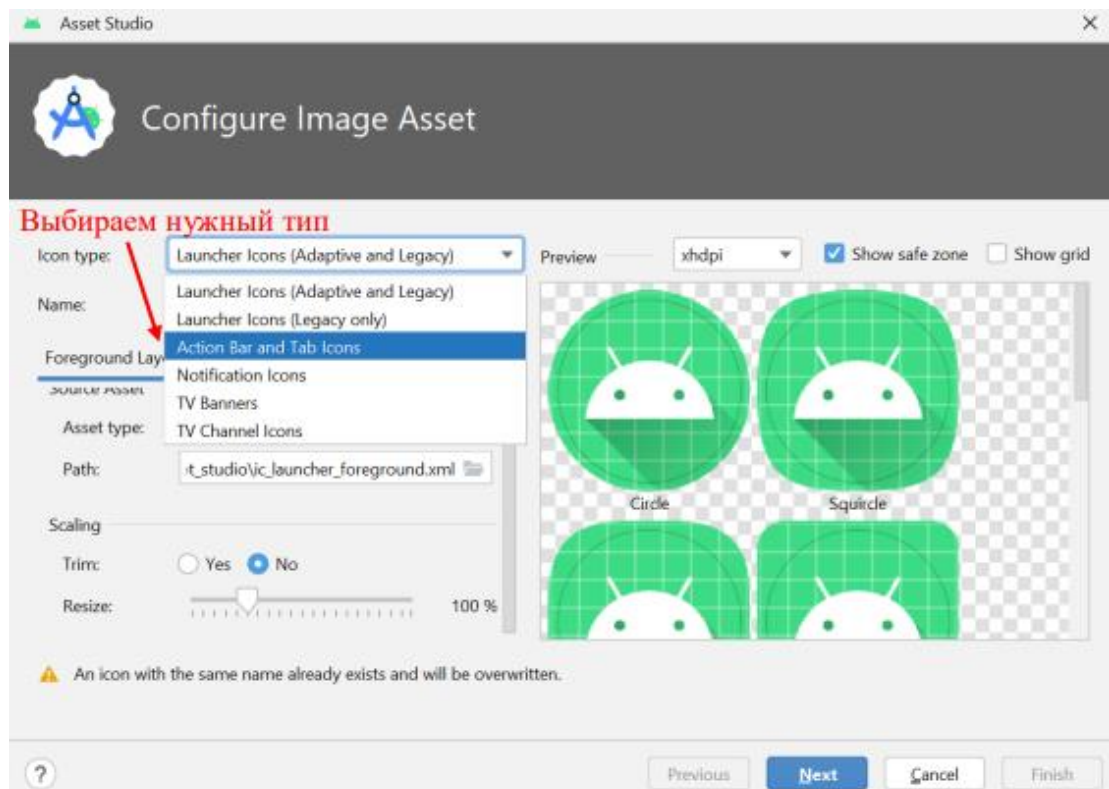


Рисунок 1.1.7 – Выбор необходимого типа иконки

В открывшемся окне пользователь может назвать значок как угодно, но он не должен содержать ни одной заглавной буквы. Пользователь может выбрать нужный значок, выполнив поиск по нему, а когда пользователь закончит, то необходимо нажать "Next" → "Finish" (рисунки 1.1.8-1.1.10).

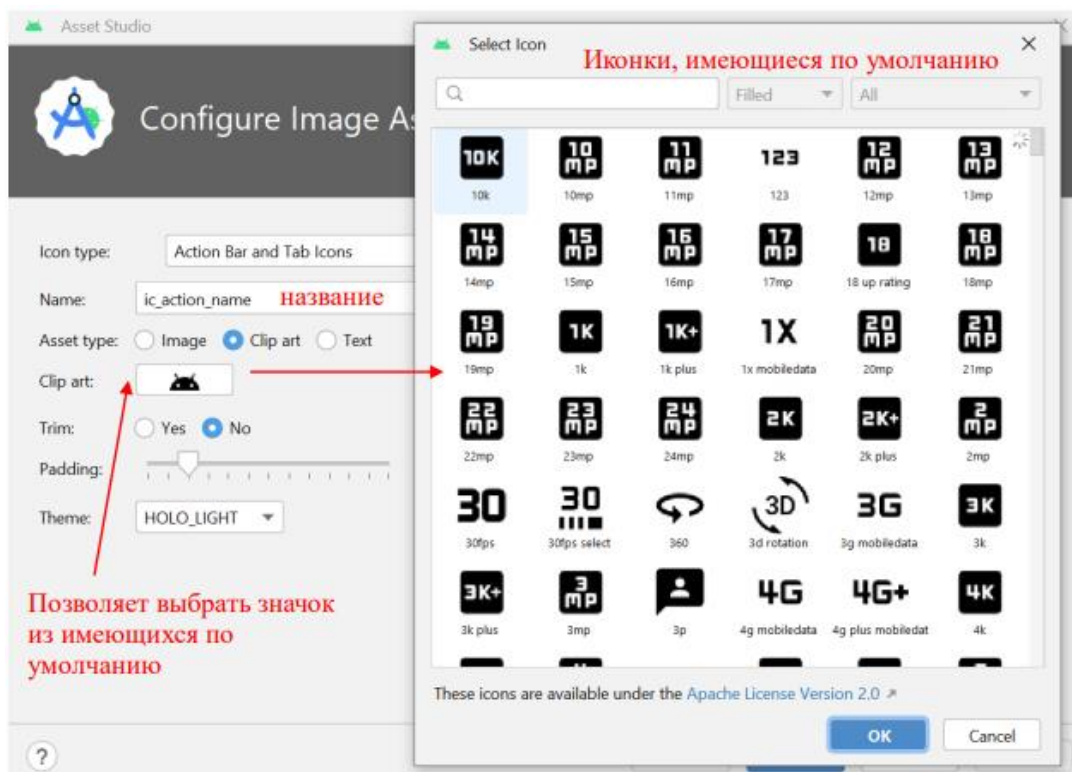


Рисунок 1.1.8 – Часть 1 настройки иконки



Рисунок 1.1.9 - Часть 2 настройки иконки

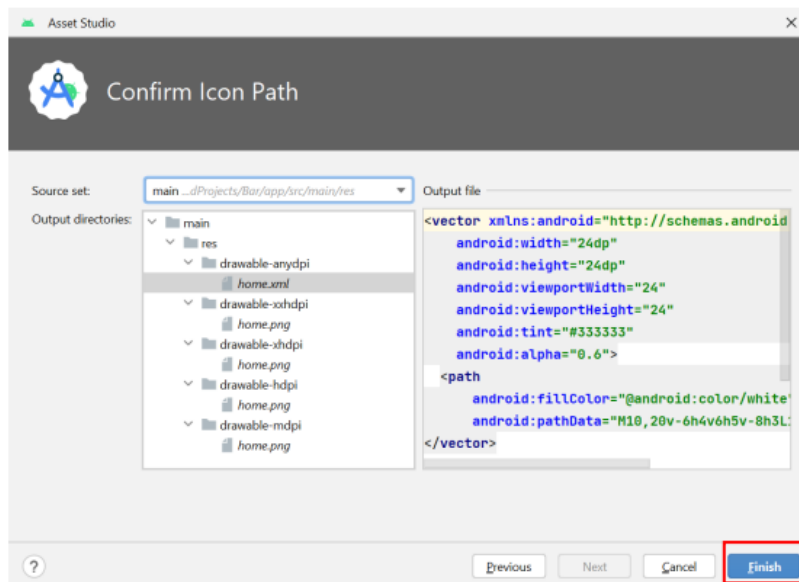


Рисунок 1.1.10 - Часть 3 настройки иконки

Image Asset поможет создать следующие типы значков:

- иконки лаунчера;
- значки панели действий и вкладок;
- значки уведомлений.

Если запустить сборку проекта, то можно получить отображение меню в нижней части экрана (рисунок 1.1.11).

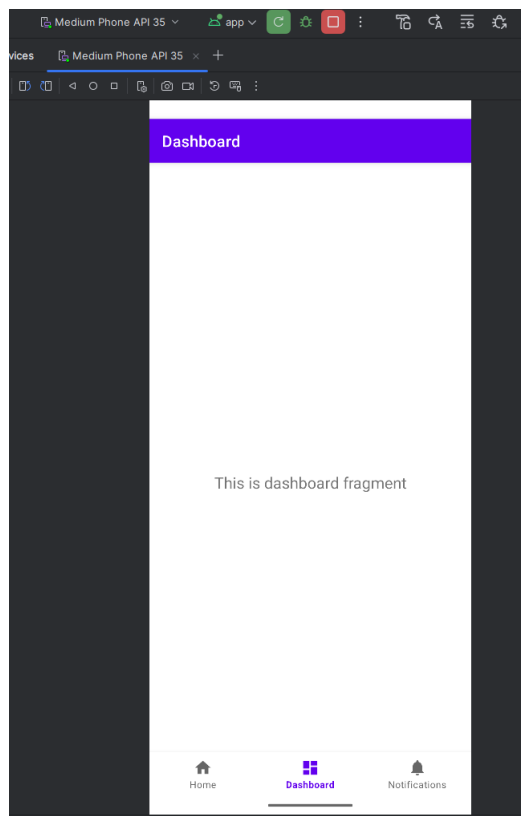


Рисунок 1.1.11 – Отображение нижнего меню в итоговом приложении

Однако только отображение не подходит для работы с BottomBar. Необходимо, чтобы при нажатии на элемент, происходили некоторые действия. Для этого необходимо установить слушатель нажатий в классе активности (рисунок 1.1.12).

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
    bottomNavigationView.setOnNavigationItemSelectedListener
        (new BottomNavigationView.OnNavigationItemSelectedListener() {
            1 usage:
            @Override
            public boolean onNavigationItemSelected(@NonNull MenuItem item) {
                if (item.getItemId() == R.id.home){
                    // Обработка выбора раздела "Домой"
                    Toast.makeText( context: MainActivity.this, text: "Домой", Toast.LENGTH_LONG).show();
                    return true;
                }
                else if (item.getItemId()==R.id.settings){
                    // Обработка выбора раздела "Настройки"
                    Toast.makeText( context: MainActivity.this, text: "Настройки", Toast.LENGTH_LONG).show();
                }
                return false;
            }
        });
}
```

Рисунок 1.1.12 – Реализация логики обработчика событий нижнего меню

Чтобы понять, какой пункт меню выбран, вначале получаем его идентификатор через `item.getItemId()`. Затем пробегаемся в конструкции `if...else` и выбираем нужный вариант и в зависимости от выбора производим определенные действия (рисунки 1.1.13-1.1.14).

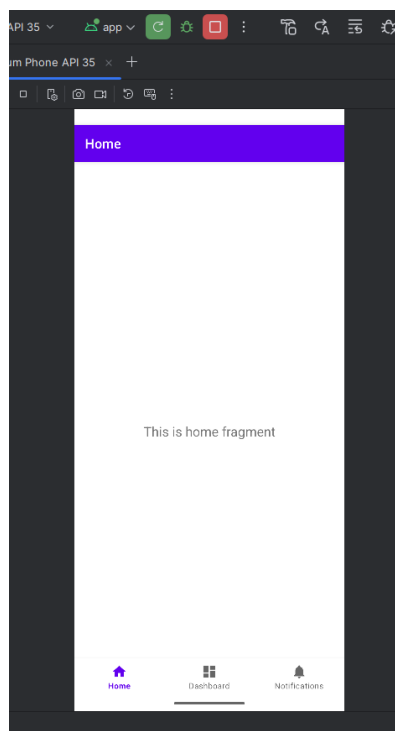


Рисунок 1.1.13 – Часть 1 отображения результата возможных событий

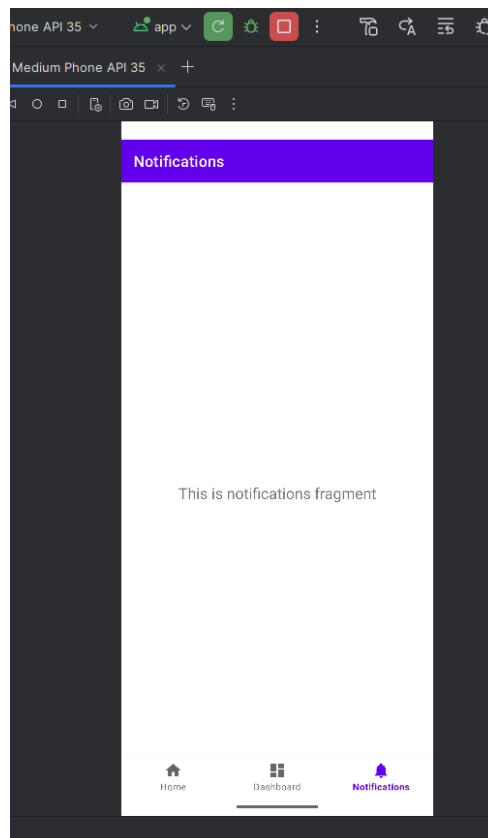


Рисунок 1.1.14 – Часть 2 отображения результата возможных событий

На рисунках 1.1.13 и 1.1.14 показан результат обработки нажатия пользователем на иконку “Notifications”.

1.2 ActionBar

ActionBar в Android представляет собой верхнюю панель приложения, которая обеспечивает удобный доступ к наиболее важным функциям приложения, а также поддерживает навигацию. Создание и использование ActionBar в связке с нижним и боковым меню может значительно улучшить пользовательский интерфейс и удобство навигации в приложении, например показывая текущий экран, на котором находится пользователь.

ActionBar уже встроен в стандартные темы Activity начиная с API Level 11 (Honeycomb). Для его использования необходимо убедиться, что выбранная Activity наследуется от AppCompatActivity, и используется одну из тем Theme.AppCompat. ActionBar будет отображаться вверху страницы (рисунок 1.2.1).

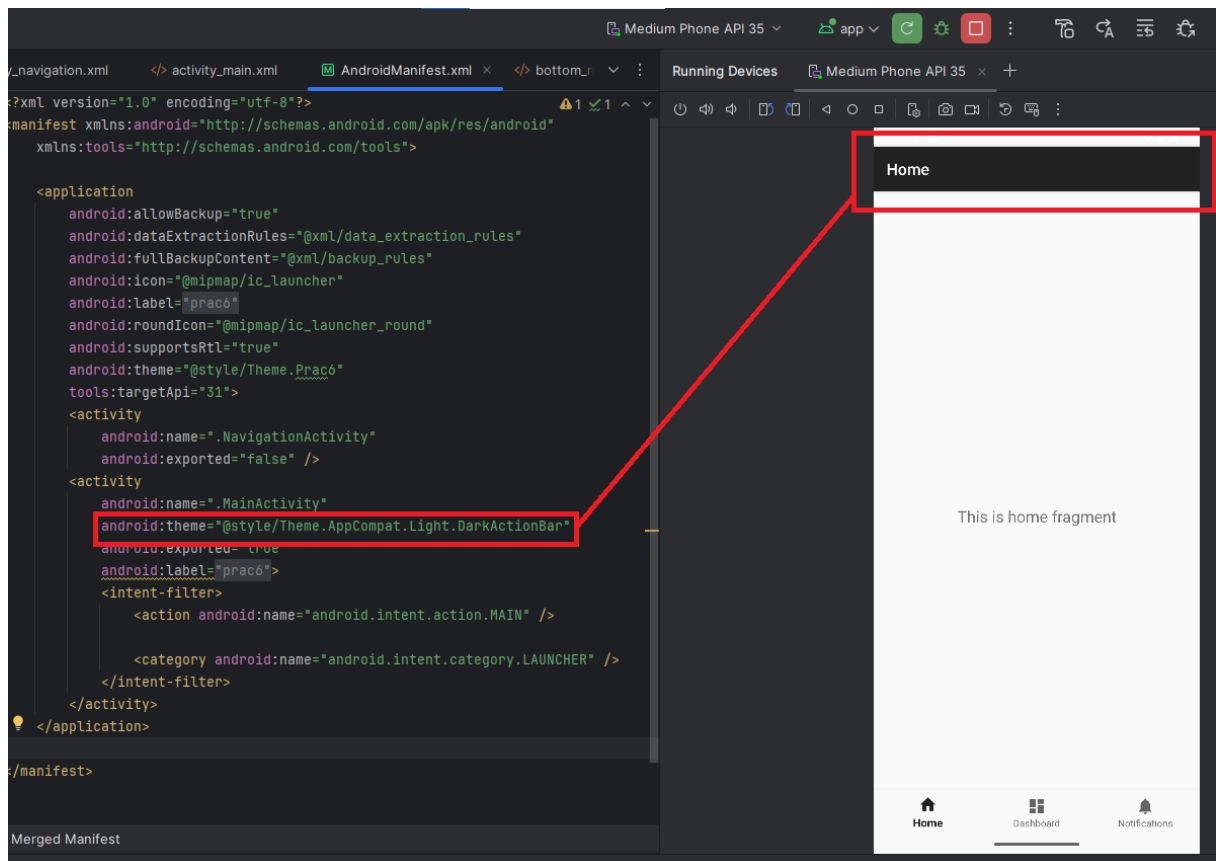


Рисунок 1.2.1 - Графический и кодовый вид ActionBar

Однако такой ActionBar не несет никакой пользы. Для дальнейшей работы его необходимо настроить. В активности можно настроить ActionBar на отображение какого-либо строкового значения (рисунок 1.2.2).

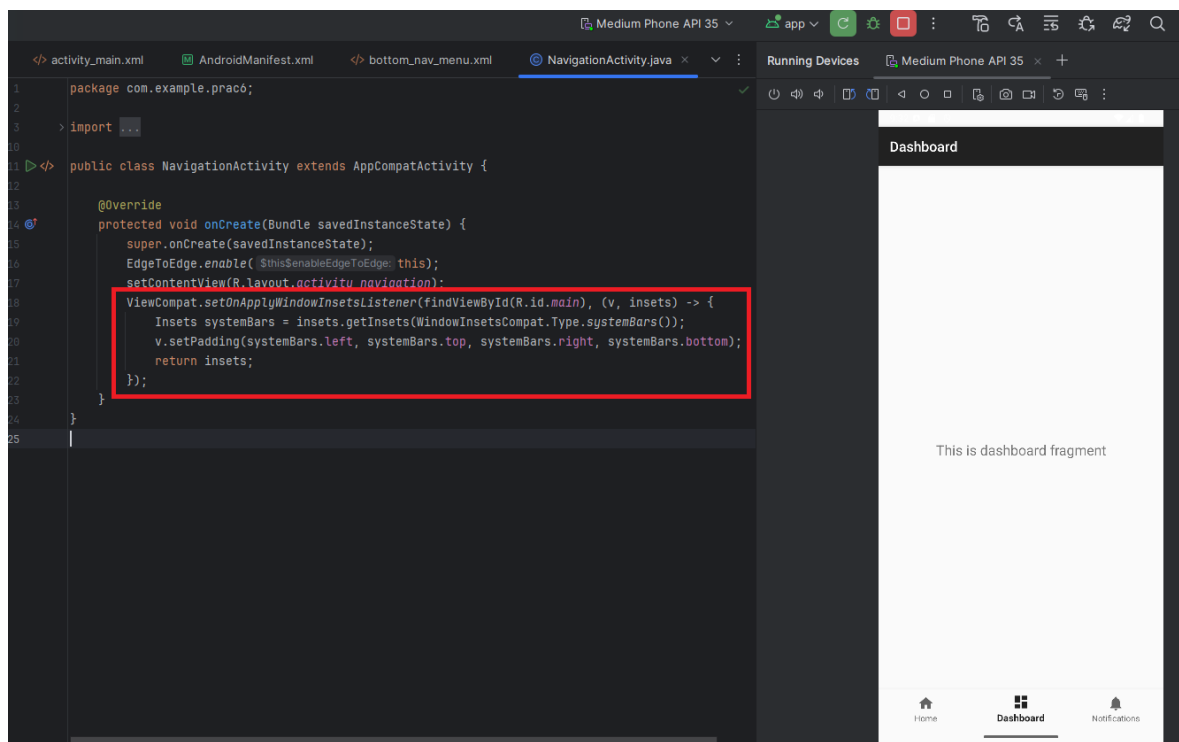


Рисунок 1.2.2 – Реализация логики обработки событий для ActionBar

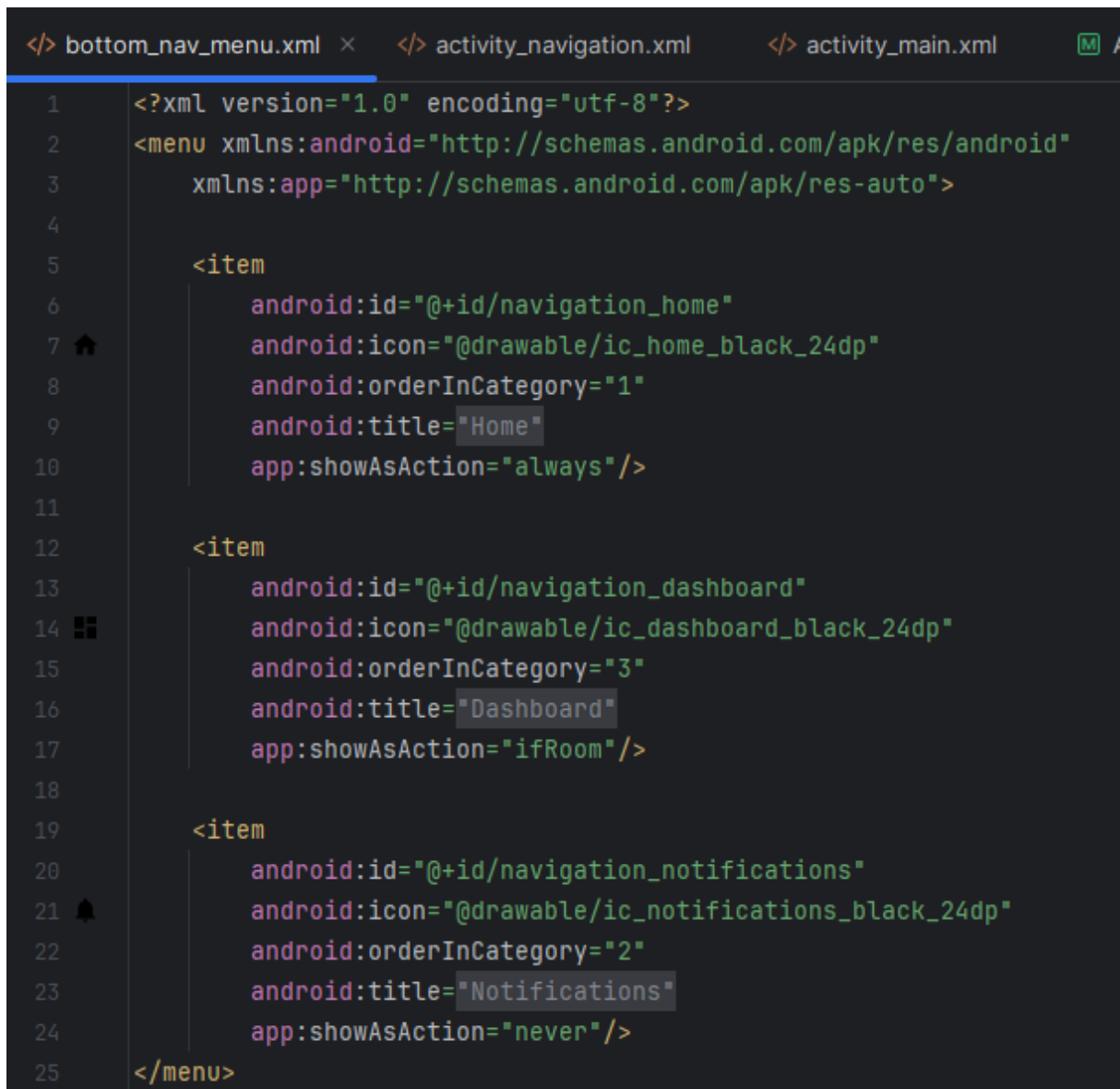
Однако этого тоже может быть недостаточно, так как нам может понадобиться быстрой доступ к некоторым элементам.

Используем ранее созданный файл `bottom_nav_menu.xml`, только немного его изменим, добавив элементам меню некоторые параметры для наглядности (рисунок 1.2.3).

- `android:orderInCategory`: значение этого атрибута определяет положение элемента в `ActionBar`. Есть два способа определить положение различных пунктов меню. Первый - предоставить одинаковое значение этого атрибута для всех элементов, и позиция будет определена в том же порядке, в каком они объявлены в коде. Второй способ - предоставить разные числовые значения для всех элементов, и тогда элементы будут располагаться в соответствии с порядком возрастания значения этого атрибута;
- `app:showAsAction`: этот атрибут определяет, как элемент будет присутствовать на панели действий.

На выбор для значения поля `app:showAsAction` предлагается четыре возможных флага:

- `always`: постоянно отображать элемент на панели действий;
- `ifRoom`: сохранить элемент, если есть свободное место;
- `never`: с этим флагом элемент не будет отображаться в виде значка в `ActionBar`, но будет присутствовать в меню переполнения;
- `withText`: чтобы представить элемент одновременно в виде значка и заголовка, можно дополнить этот флаг флагом `always` или `ifRoom` (`always|withText` or `ifRoom|withText`).



```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

    <item
        android:id="@+id/navigation_home"
        android:icon="@drawable/ic_home_black_24dp"
        android:orderInCategory="1"
        android:title="Home"
        app:showAsAction="always"/>

    <item
        android:id="@+id/navigation_dashboard"
        android:icon="@drawable/ic_dashboard_black_24dp"
        android:orderInCategory="3"
        android:title="Dashboard"
        app:showAsAction="ifRoom"/>

    <item
        android:id="@+id/navigation_notifications"
        android:icon="@drawable/ic_notifications_black_24dp"
        android:orderInCategory="2"
        android:title="Notifications"
        app:showAsAction="never"/>

</menu>
```

Рисунок 1.2.3 – Изменённый файл разметки bottom_nav_menu.xml

На рисунке 1.2.3 определено меню с тремя элементами, но само определение элементов в файле еще не создает меню. Это всего лишь описание. Чтобы вывести его на экран, надо использовать его в классе Activity. Для этого надо переопределить метод onCreateOptionsMenu.

Метод getMenuInflater возвращает объект MenuInflater, у которого вызывается метод inflate(). Этот метод в качестве первого параметра принимает ресурс, представляющий наше описание меню в xml, и наполняет им объект menu, переданный в качестве второго параметра.

Теперь, если мы нажмем на любой из пунктов меню, то ничего не произойдет. Чтобы привязать к меню действия, нам надо переопределить в классе activity метод onOptionsItemSelected (рисунок 1.2.4).

```

19 bottom_nav_menu.xml </> activity_navigation.xml </> activity_main.xml MainActivity.java x AndroidManifest.xml Navigation
20
21 public class MainActivity extends AppCompatActivity {
22     protected void onCreate(Bundle savedInstanceState) {
23
24         binding = ActivityMainBinding.inflate(getLayoutInflater());
25         setContentView(binding.getRoot());
26
27         BottomNavigationView navView = findViewById(R.id.nav_view);
28         // Passing each menu ID as a set of IDs because each
29         // menu should be considered as top level destinations.
30         AppBarConfiguration appBarConfiguration = new AppBarConfiguration.Builder(
31             R.id.navigation_home, R.id.navigation_dashboard, R.id.navigation_notifications)
32             .build();
33         NavController navController = Navigation.findNavController(this, R.id.nav_host_fragment_activity_main);
34         NavigationUI.setupActionBarWithNavController(this, navController, appBarConfiguration);
35         NavigationUI.setupWithNavController(binding.navView, navController);
36     }
37
38     @Override
39     public boolean onCreateOptionsMenu(Menu menu) {
40         getMenuInflater().inflate(R.menu.bottom_nav_menu, menu);
41         return super.onCreateOptionsMenu(menu);
42     }
43
44     @Override
45     public boolean onOptionsItemSelected(@NonNull MenuItem item) {
46         //определение нажатого элемента
47         if (item.getItemId() == R.id.navigation_home) {
48             // обработка выбора раздела "Домой"
49             Toast.makeText(context, MainActivity.this, text: "Домой", Toast.LENGTH_LONG).show();
50             return true;
51         } else if (item.getItemId() == R.id.navigation_notifications) {
52             // обработка выбора раздела "Настройки"
53             Toast.makeText(context, MainActivity.this, text: "Настройки", Toast.LENGTH_LONG).show();
54         }
55         return super.onOptionsItemSelected(item);
56     }
57 }

```

Рисунок 1.2.4 – Переопределение необходимых методов для ActionBar

И при нажатии на троеточие в правом верхнем углу экрана, а после нажатия на “Notifications” появится сообщение “Notifications” (рисунок 1.2.5).

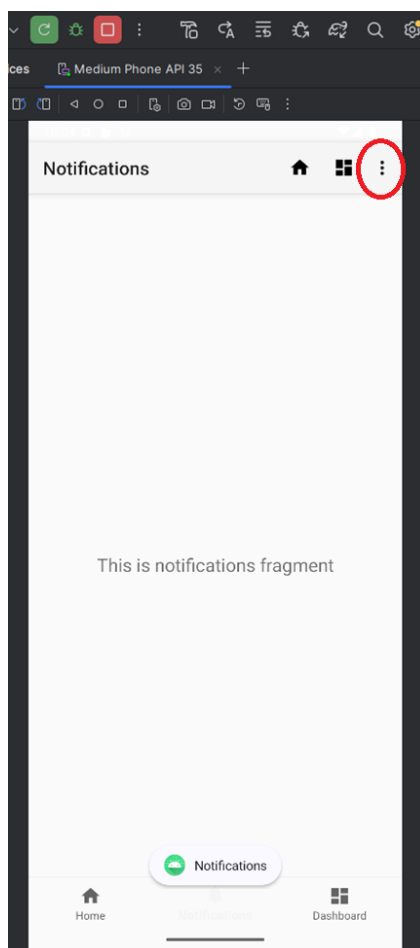
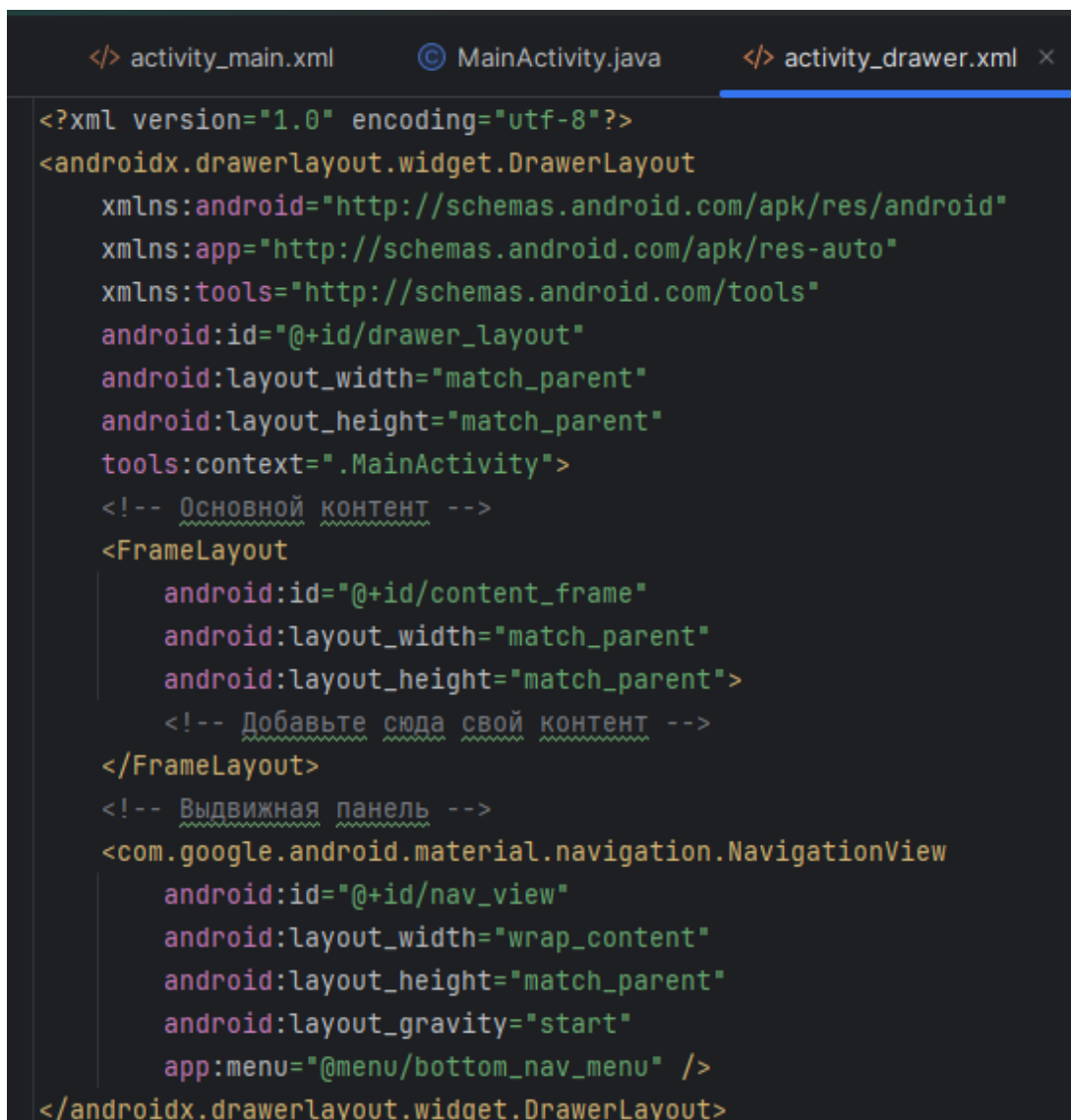


Рисунок 1.2.5 – Итоговый вид ActionBar-а в рабочем приложении

Часть 1.3 Navigation Drawer

Создание выдвигной панели (Drawer) в Android обычно реализуется с помощью DrawerLayout и NavigationView. DrawerLayout используется как корневой контейнер интерфейса, который позволяет разместить основное содержимое приложения и выдвигную панель. NavigationView представляет собой панель навигации, которая отображается в DrawerLayout и содержит элементы меню для навигации по различным разделам приложения.

В файле макета активности необходимо добавить DrawerLayout как корневой элемент. Внутри DrawerLayout разместим основной контент и NavigationView для выдвигной панели (рисунок 1.3.1).



```
<?xml version="1.0" encoding="utf-8"?>
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <!-- Основной контент -->
    <FrameLayout
        android:id="@+id/content_frame"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <!-- Добавьте сюда свой контент -->
    </FrameLayout>
    <!-- Выдвижная панель -->
    <com.google.android.material.navigation.NavigationView
        android:id="@+id/nav_view"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start"
        app:menu="@menu/bottom_nav_menu" />
</androidx.drawerlayout.widget.DrawerLayout>
```

Рисунок 1.3.1 – Кодовый вид файла разметки activity_drawer.xml

Создаётся XML-файл в папке res/menu (например, drawer_menu.xml) и добавьте пункты меню для вашей выдвижной панели (рисунок 1.3.2).

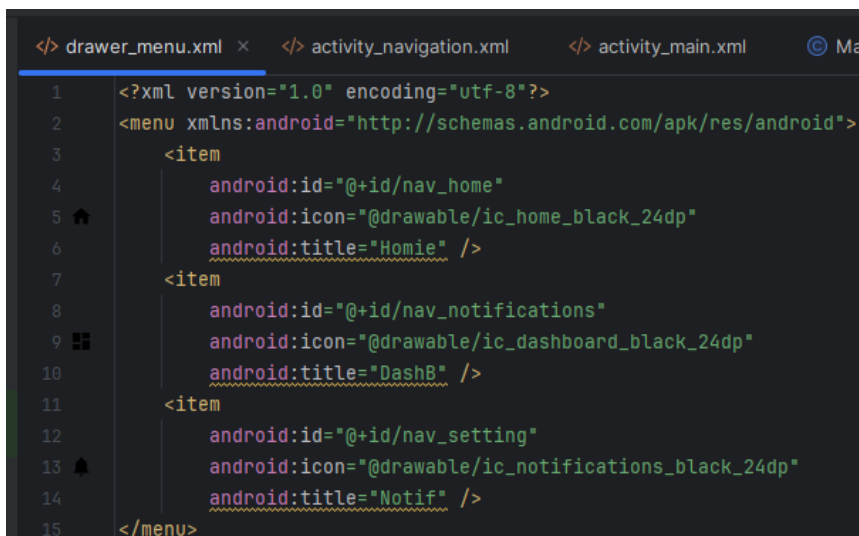


Рисунок 1.3.2 – Описание элементов файла drawer_menu.xml

Затем по аналогии с BottomBar создать слушатель нажатий для каждого элемента меню.

Для интеграции ActionBar с DrawerLayout используется ActionBarDrawerToggle, который добавит иконку меню для открытия и закрытия Drawer и обеспечит анимацию иконки (рисунок 1.3.3).

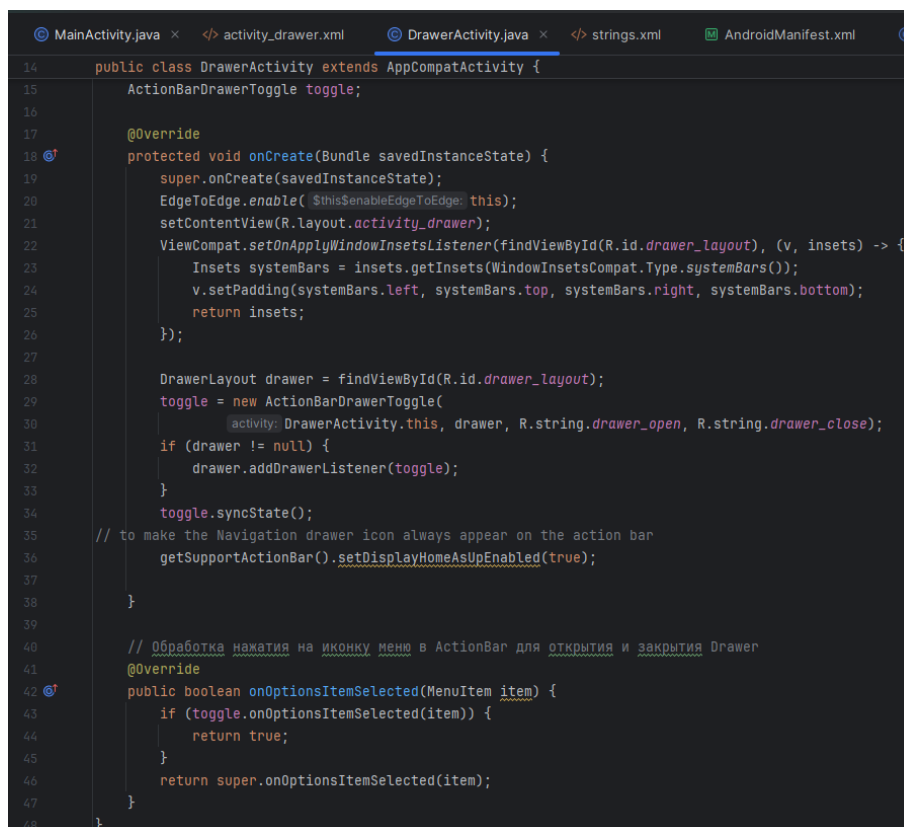


Рисунок 1.3.3 – Реализация логики Navigation Drawer

Также на рисунке 1.3.3 был добавлен слушатель для обработки нажатия на элементы (onOptionsItemSelected).

В итоге будет отображаться всплывающее меню с выбранными строковыми значениями (рисунки 1.3.4-1.3.5).



Рисунок 1.3.4 – Часть 1 отображения всплывающего меню

После запуска приложения пользователь нажмёт на иконку всплывающего меню, находящуюся в левом верхнем углу, и увидит содержимое всплывающего меню (рисунок 1.3.5).

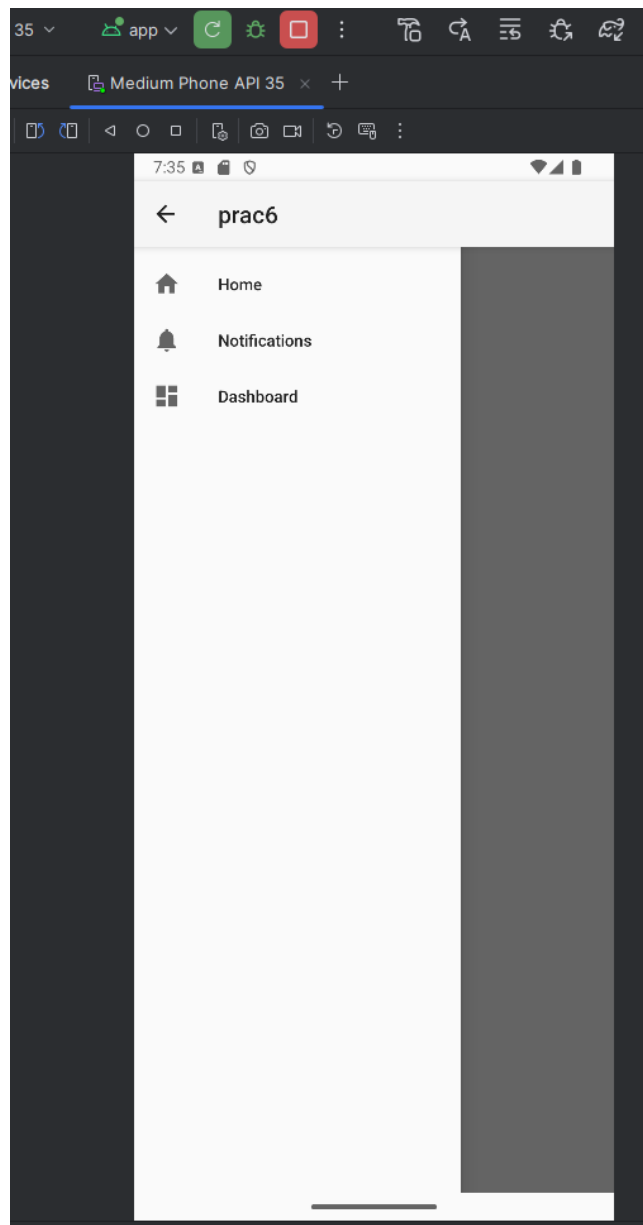


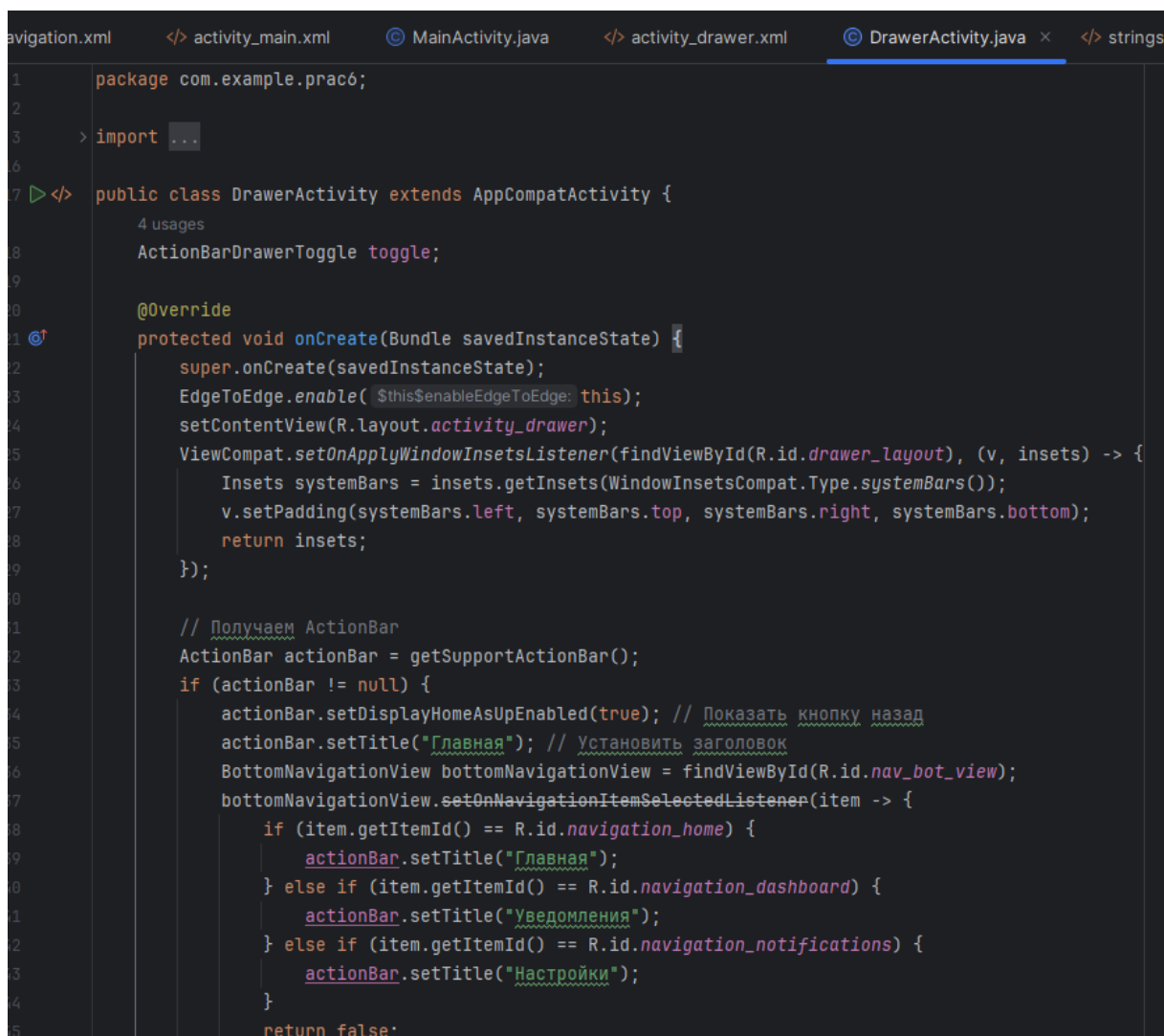
Рисунок 1.3.5 - Часть 2 отображения всплывающего меню

Чтобы использовать `ActionBarDrawerToggle`, нужно создать экземпляр при помощи конструктора со следующими аргументами:

- Activity, в котором размещается боковая панель навигации;
- `DrawerLayout` – drawable ресурс, используемый в качестве индикатора панели. Стандартный навигационный значок панели доступен в `Download the Action Bar Icon Pack`;
- Строковый ресурс для обозначения открытой панели (для специальных возможностей);
- Строковый ресурс для обозначения закрытой панели (для специальных возможностей).

Теперь, в зависимости от использования класса ActionBarDrawerToggle в списке панели навигации, необходимо вызвать ActionBarDrawerToggle в нескольких местах жизненного цикла activity.

Кроме этого, ActionBar можно использовать вместе с BottomNavigationView для создания единой системы навигации. В этом случае ActionBar обычно служит для отображения контекстной информации (например, заголовка текущей страницы), а BottomNavigationView для навигации между основными разделами приложения (рисунки 1.3.6-1.3.7).



```
1 package com.example.prac6;
2
3 > import ...
4
5
6
7 </> public class DrawerActivity extends AppCompatActivity {
8     4 usages
9     ActionBarDrawerToggle toggle;
10
11     @Override
12     protected void onCreate(Bundle savedInstanceState) {
13         super.onCreate(savedInstanceState);
14         EdgeToEdge.enable(this);
15         setContentView(R.layout.activity_drawer);
16         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.drawer_layout), (v, insets) -> {
17             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
18             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
19             return insets;
20         });
21
22         // Получаем ActionBar
23         ActionBar actionBar = getSupportActionBar();
24         if (actionBar != null) {
25             actionBar.setDisplayHomeAsUpEnabled(true); // Показать кнопку назад
26             actionBar.setTitle("Главная"); // Установить заголовок
27             BottomNavigationView bottomNavigationView = findViewById(R.id.nav_bot_view);
28             bottomNavigationView.setOnNavigationItemSelectedListener(item -> {
29                 if (item.getItemId() == R.id.navigation_home) {
30                     actionBar.setTitle("Главная");
31                 } else if (item.getItemId() == R.id.navigation_dashboard) {
32                     actionBar.setTitle("Уведомления");
33                 } else if (item.getItemId() == R.id.navigation_notifications) {
34                     actionBar.setTitle("Настройки");
35                 }
36             });
37         }
38         return false;
39     }
40 }
```

Рисунок 1.3.6 – Часть 1 реализации логики BottomNavigationView

```

_navigation.xml    <> activity_main.xml    MainActivity.java    <> activity_drawer.xml    DrawerActivity.java    <> strin
17    public class DrawerActivity extends AppCompatActivity {
21        protected void onCreate(Bundle savedInstanceState) {
37            bottomNavigationView.setOnNavigationItemSelectedListener(item -> {
40                } else if (item.getItemId() == R.id.navigation_dashboard) {
41                    actionBar.setTitle("Уведомления");
42                } else if (item.getItemId() == R.id.navigation_notifications) {
43                    actionBar.setTitle("Настройки");
44                }
45                return false;
46            });
47        }
48        DrawerLayout drawer = findViewById(R.id.drawer_layout);
49        toggle = new ActionBarDrawerToggle(
50            activity, DrawerActivity.this, drawer, R.string.drawer_open, R.string.drawer_close);
51        if (drawer != null) {
52            drawer.addDrawerListener(toggle);
53        }
54        toggle.syncState();
55        // to make the Navigation drawer icon always appear on the action bar
56        getSupportActionBar().setDisplayHomeAsUpEnabled(true);
57    }
58    // обработка нажатия на иконку меню в ActionBar для открытия и закрытия Drawer
59    @Override
60    public boolean onOptionsItemSelected(MenuItem item) {
61        if (toggle.onOptionsItemSelected(item)) {
62            return true;
63        }
64        return super.onOptionsItemSelected(item);
65    }
66    }

```

Рисунок 1.3.7 - Часть 2 реализации логики BottomNavigationView

У пользователя появится возможность переходить по нижней панели на различные фрагменты (рисунки 1.3.8-1.3.9).

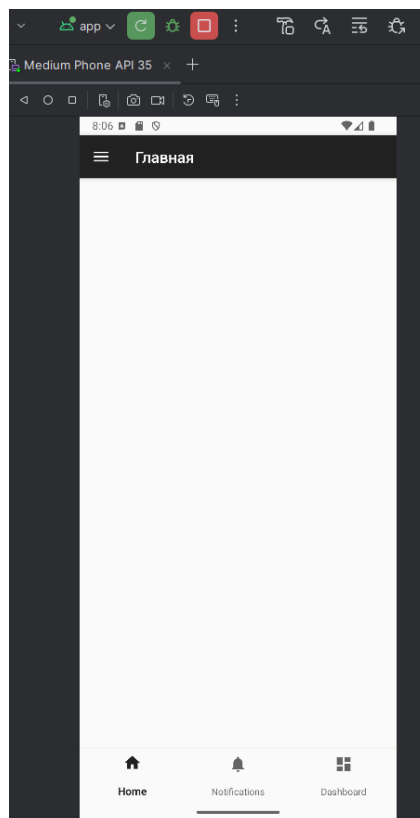


Рисунок 1.3.8 – Часть 1 отображения итогового вида приложения

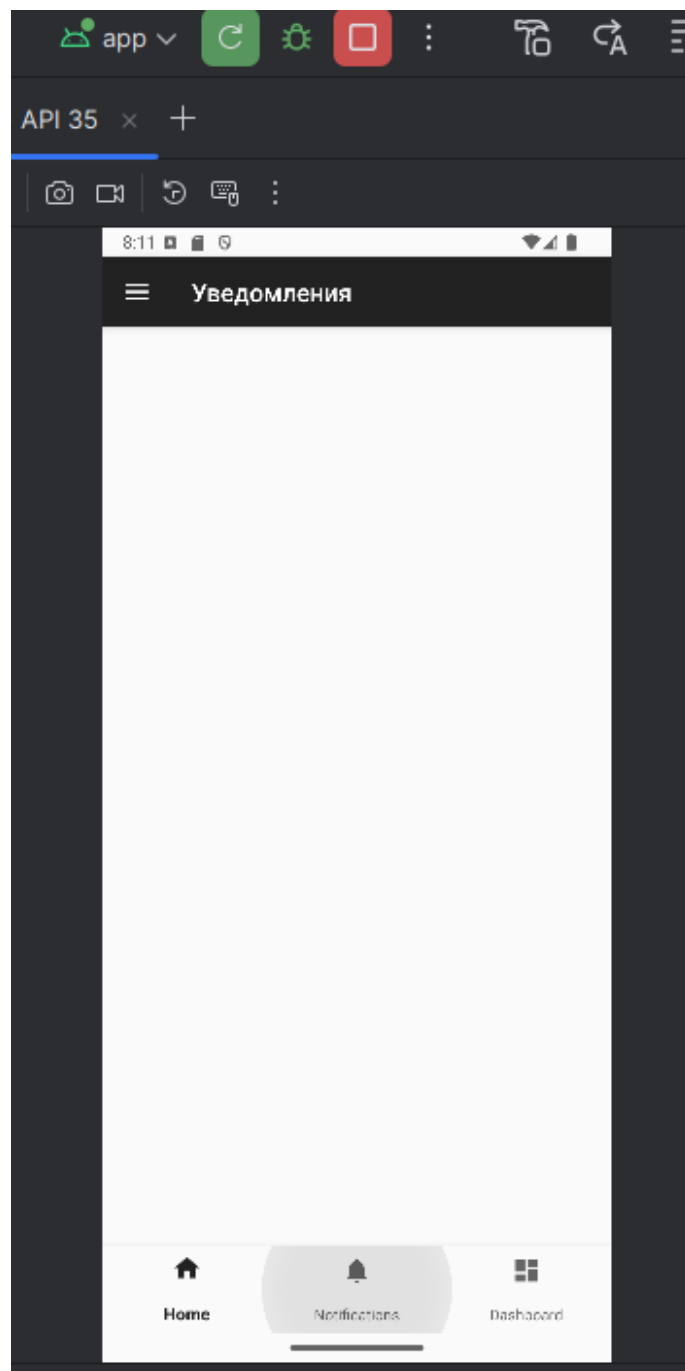


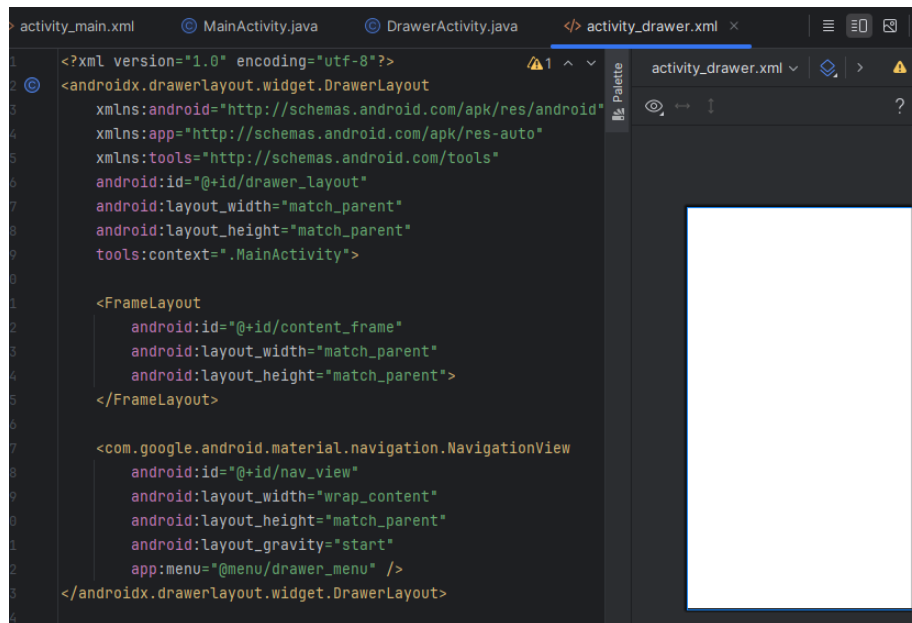
Рисунок 1.3.9 – Часть 2 отображения итогового вида приложения

Это позволяет пользователю быстро переключаться между разделами приложения с помощью нижнего меню, в то время как верхняя панель (ActionBar) отображает контекстную информацию о выбранном разделе или предоставляет дополнительные опции действий, связанных с текущим экраном. Использование ActionBar в сочетании с боковым и нижним меню позволяет создать гибкую и интуитивно понятную систему навигации для пользователей, обеспечивая легкий доступ ко всем важным функциям в приложении.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

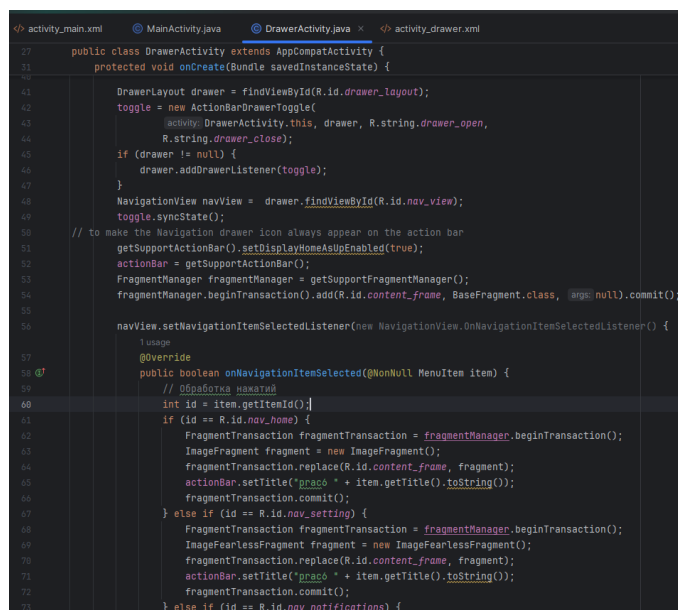
2.1 Создание интерактивного каталога в всплывающем меню

Для реализации заданного функционала были использованы такие layout-ы, как `FrameLayout` и `NavigationView` (рисунок 2.1.1).



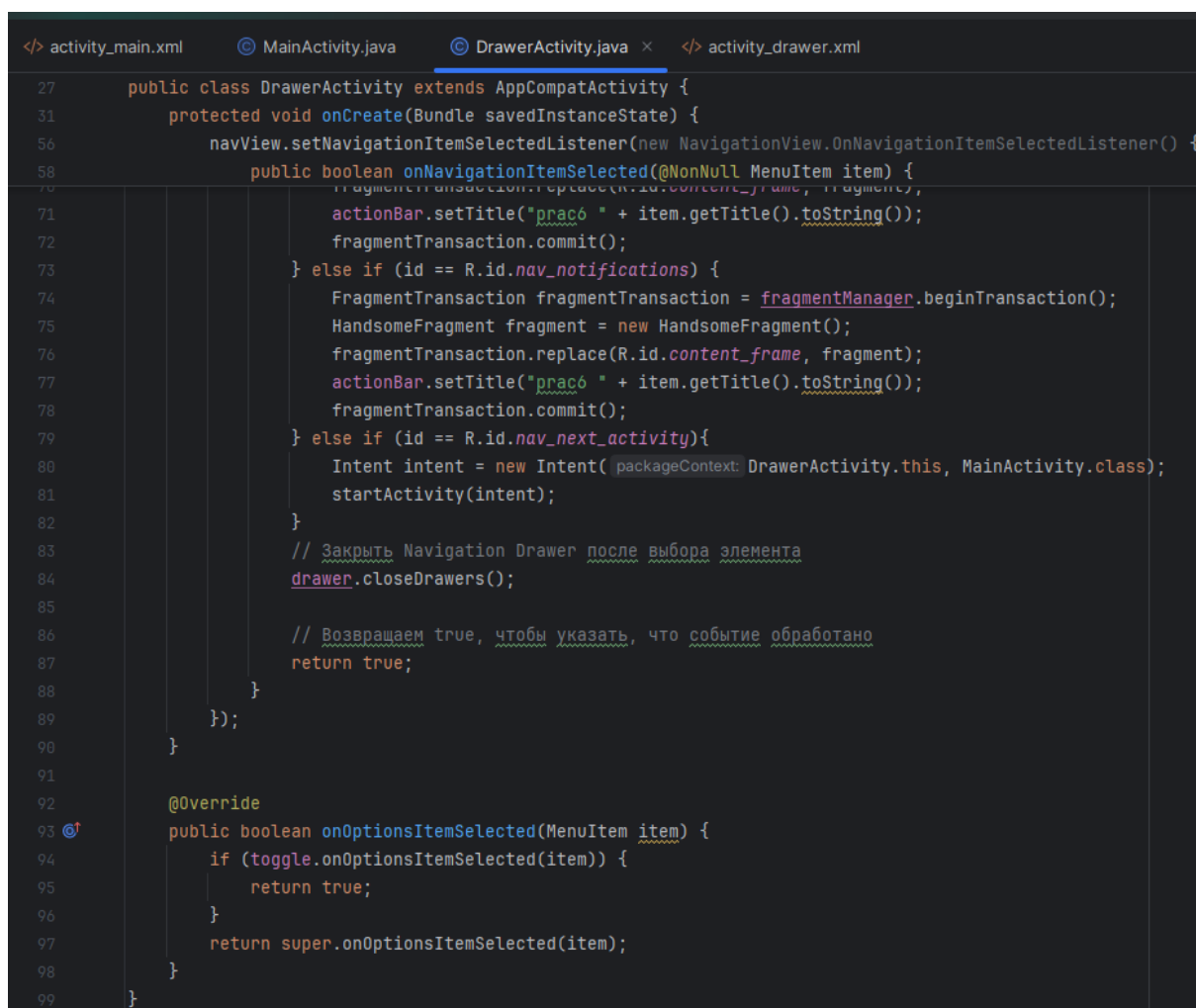
2.1.1 - Графический и кодовый вид activity_drawer.xml

При этом была реализована логика перехода на различные фрагменты в зависимости от выбора во всплывающем меню (рисунки 2.1.2-2.1.3).



2.1.2 – Часть 1 реализации логики всплывающего меню

На рисунке 2.1.2 отображено создание всех необходимых полей и начало обработки событий, происходящих во всплывающем меню.



```
27 public class DrawerActivity extends AppCompatActivity {
31     protected void onCreate(Bundle savedInstanceState) {
56         NavController navController = getSupportFragmentManager().findNavController();
58         public boolean onNavigationItemSelectedListener(@NonNull MenuItem item) {
70             FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
71             actionBar.setTitle("prac6 " + item.getTitle().toString());
72             transaction.commit();
73         } else if (id == R.id.nav_notifications) {
74             FragmentTransaction fragmentTransaction = fragmentManager.beginTransaction();
75             HandsomeFragment fragment = new HandsomeFragment();
76             fragmentTransaction.replace(R.id.content_frame, fragment);
77             actionBar.setTitle("prac6 " + item.getTitle().toString());
78             fragmentTransaction.commit();
79         } else if (id == R.id.nav_next_activity) {
80             Intent intent = new Intent(packageContext, DrawerActivity.this, MainActivity.class);
81             startActivity(intent);
82         }
83         // Закрыть Navigation Drawer после выбора элемента
84         drawer.closeDrawers();
85
86         // Возвращаем true, чтобы указать, что событие обработано
87         return true;
88     }
89 }
90
91
92 @Override
93 public boolean onOptionsItemSelected(MenuItem item) {
94     if (toggle.onOptionsItemSelected(item)) {
95         return true;
96     }
97     return super.onOptionsItemSelected(item);
98 }
99 }
```

Рисунок 2.1.3 - Часть 2 реализации логики всплывающего меню

На рисунке 2.1.3 отображено окончание обработки событий, происходящих во всплывающем меню, а также метод закрытия и открытия этого самого меню (onOptionsItemSelected).

Пользователь сможет зайти во всплывающее меню и выбрать необходимый элемент для его непосредственного отображения на экране (рисунки 2.1.4-2.1.8)

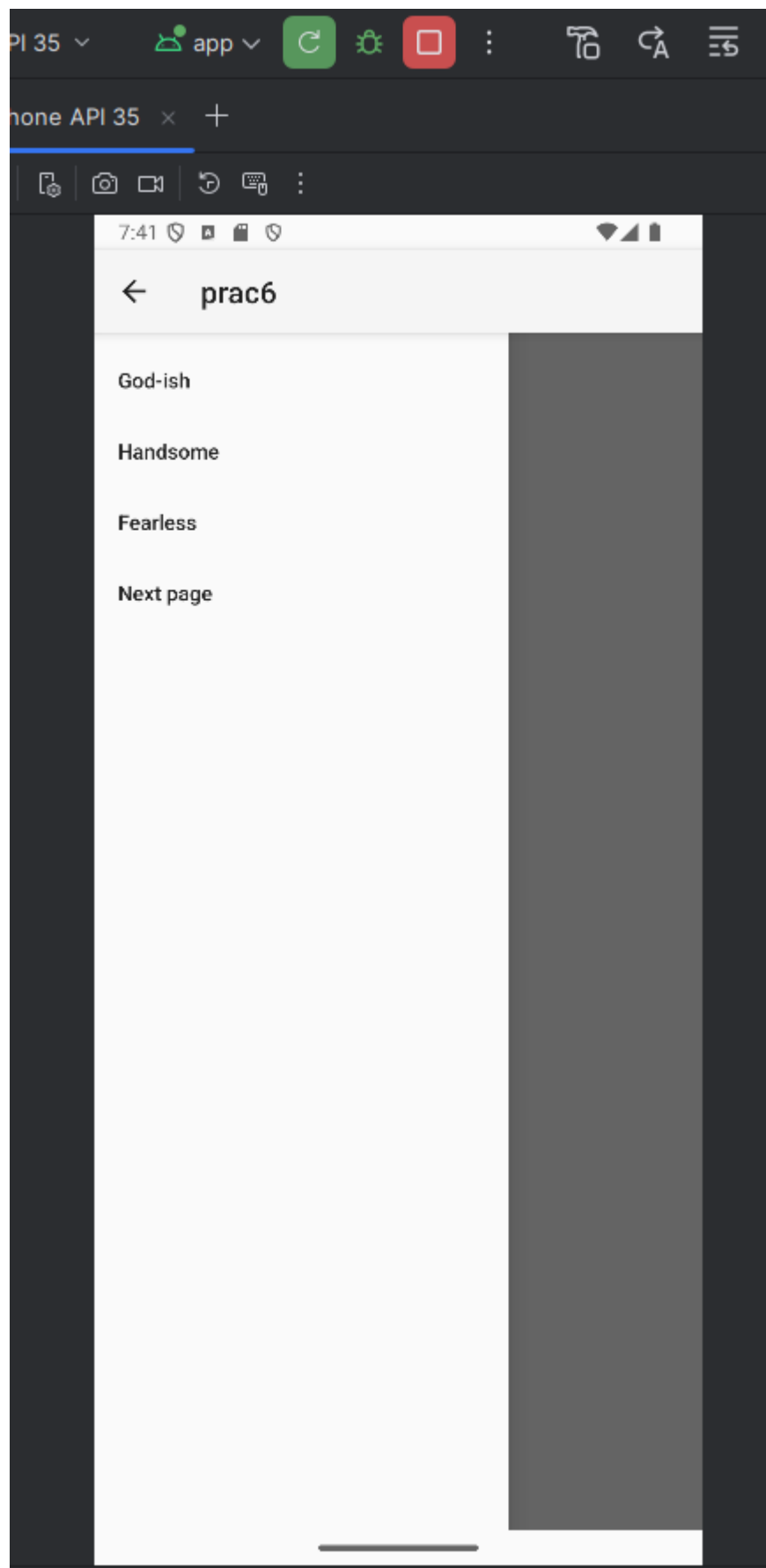
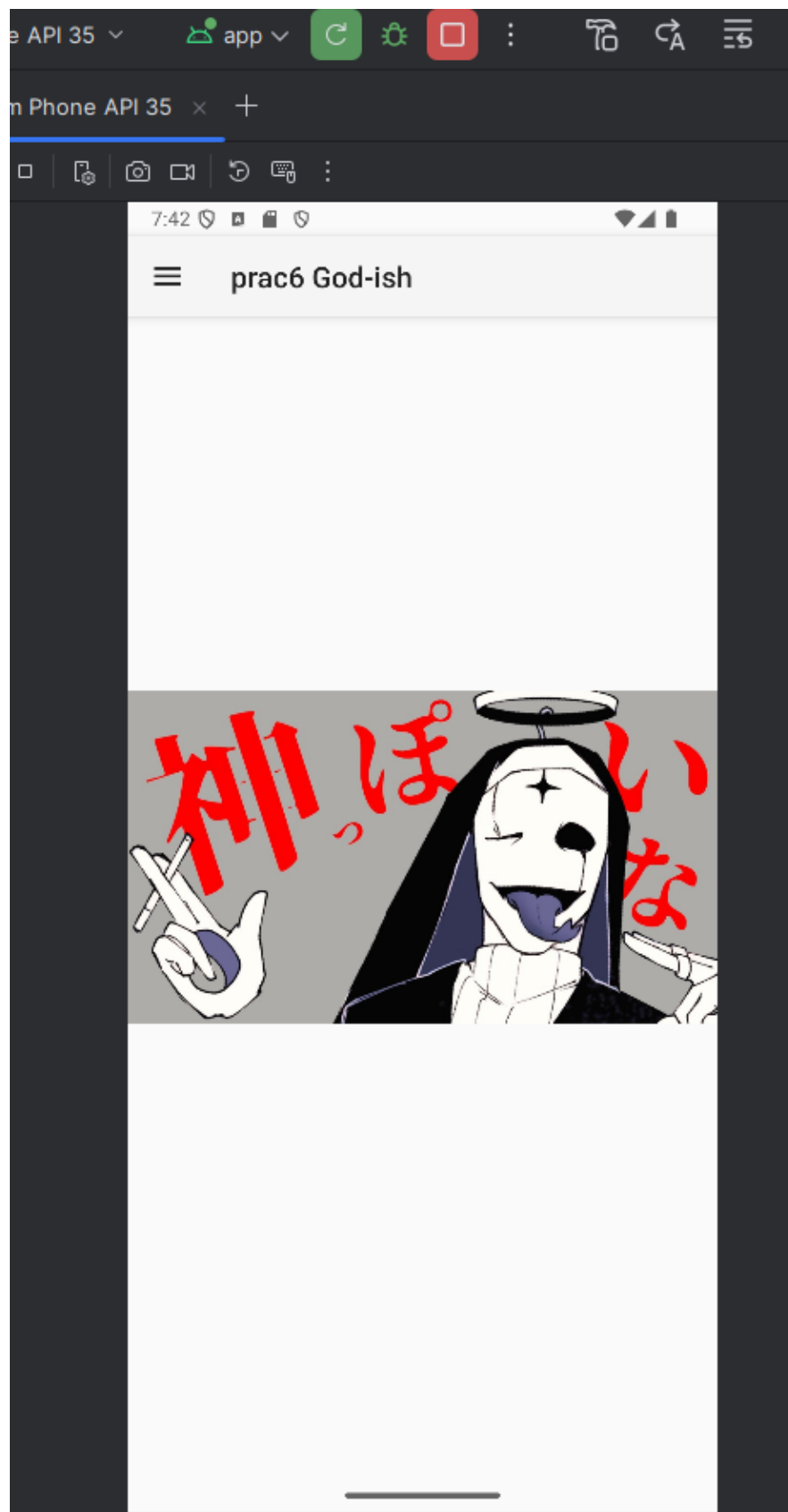


Рисунок 2.1.4 – Открытое всплывающее меню



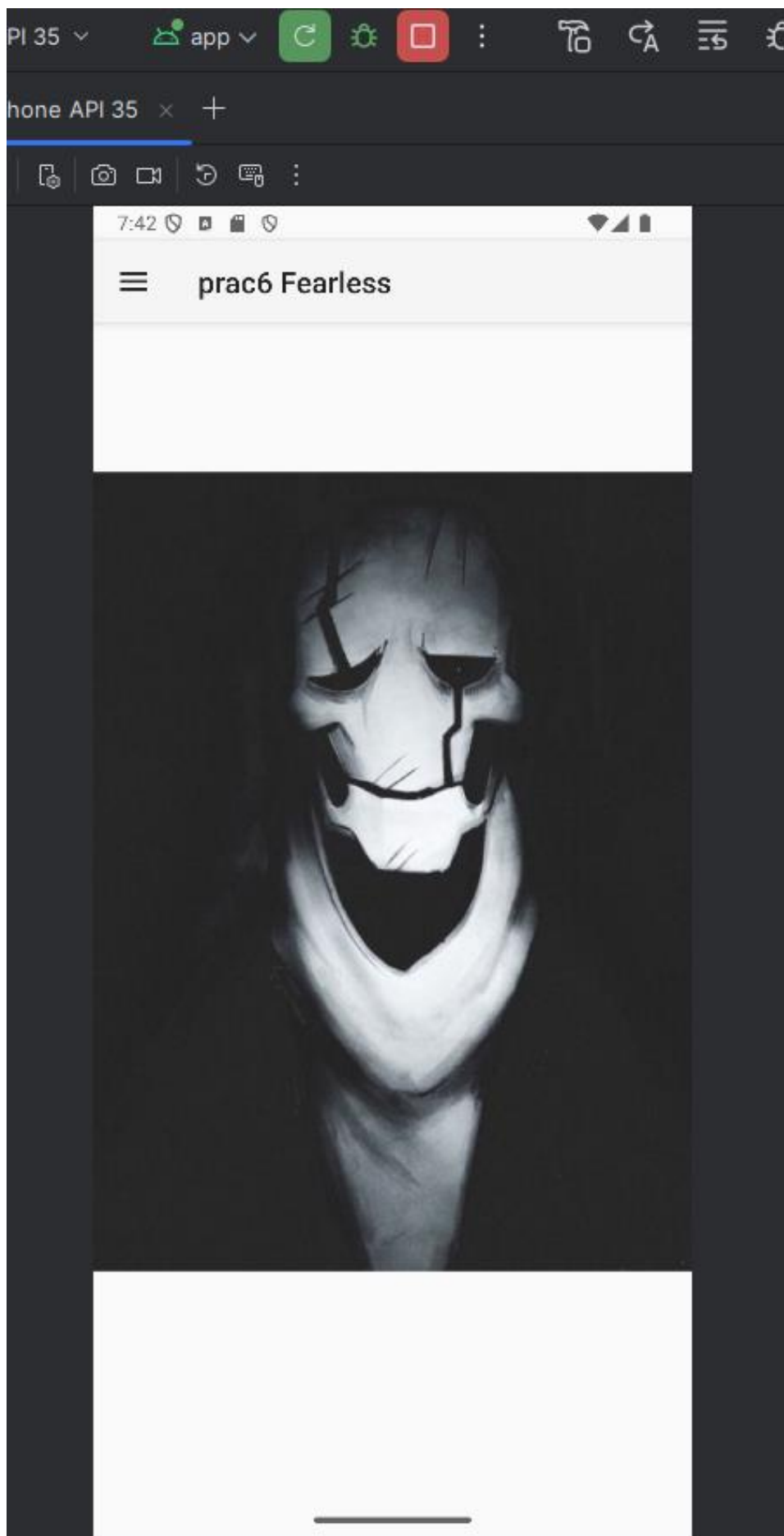
2.1.5 – Результат нажатия пользователем на элемент God-ish

На рисунке 2.1.5 отображено изменение заголовка текущей activity, а также перемещение на фрагмент, соответствующий элементу, выбранному из каталога всплывающего меню.

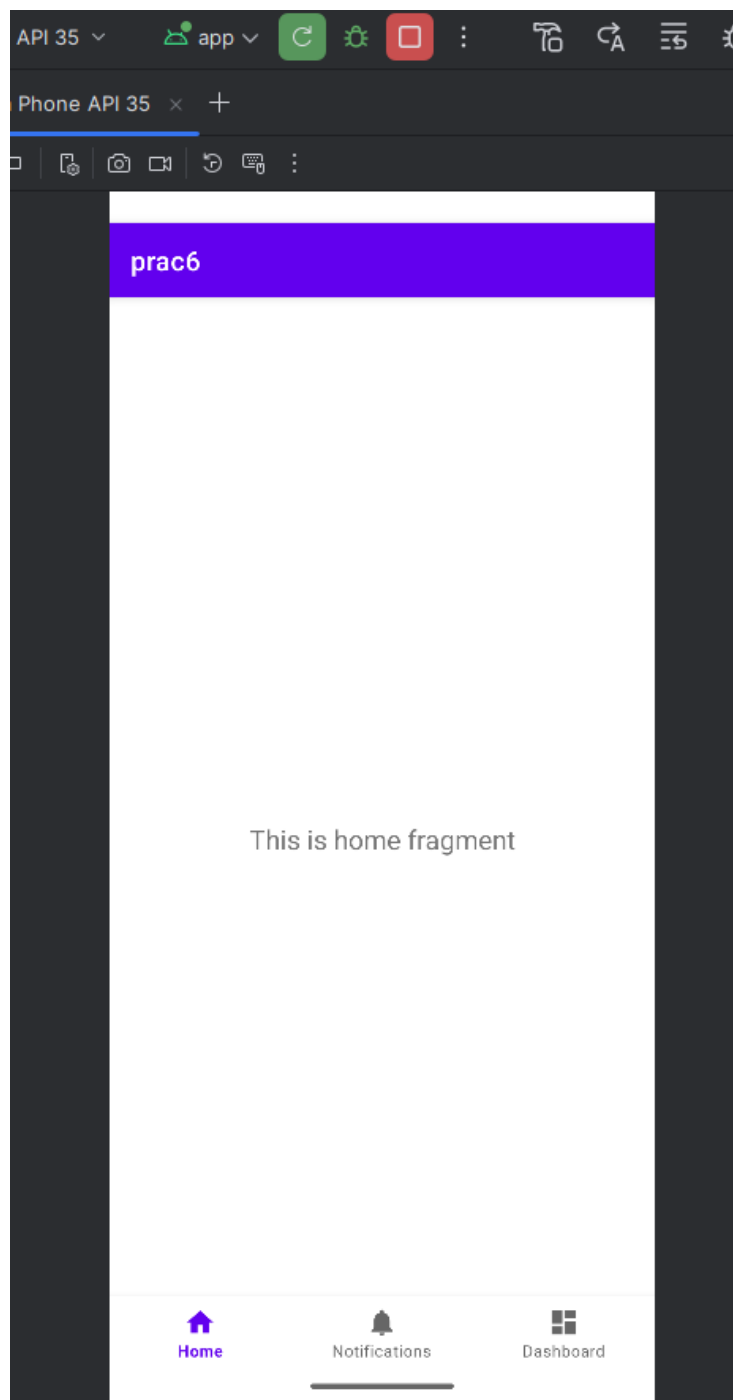


2.1.6 – Результат нажатия пользователем на элемент Handsome

Тот же самый процесс, что был упомянут для рисунка 2.1.5, отображён на рисунке 2.1.6



2.1.7 – Результат нажатия пользователем на элемент Fearless



2.1.8 – Результат нажатия пользователем на элемент Next page

Процесс, проводимый на рисунке 2.1.8, отличается от остальных: в нём происходит переход на следующую activity.

2.2 Реализация интеграции BottomBar с ActionBar

Для интеграции BottomBar с ActionBar необходимо задать соответствующие элементы в файле разметки (рисунок 2.2.1).

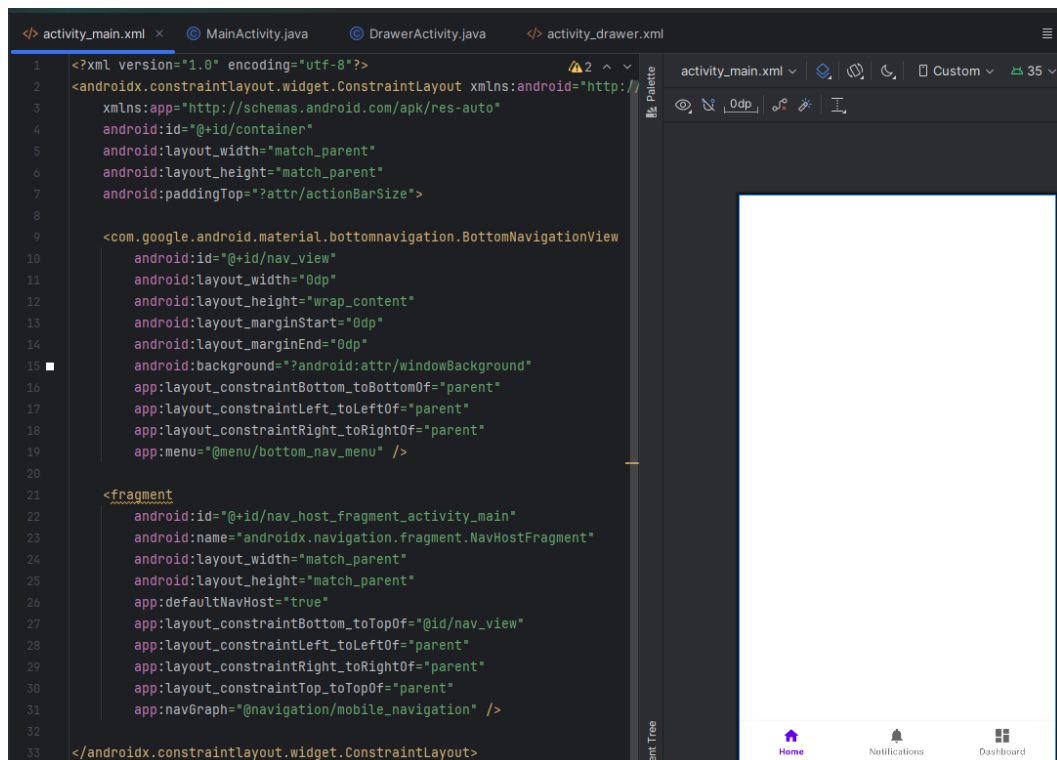


Рисунок 2.2.1 – Графический и кодовый вид activity_main.xml

Однако ActionBar будет задан в файле MainActivity.java при помощи кода (рисунок 2.2.2).

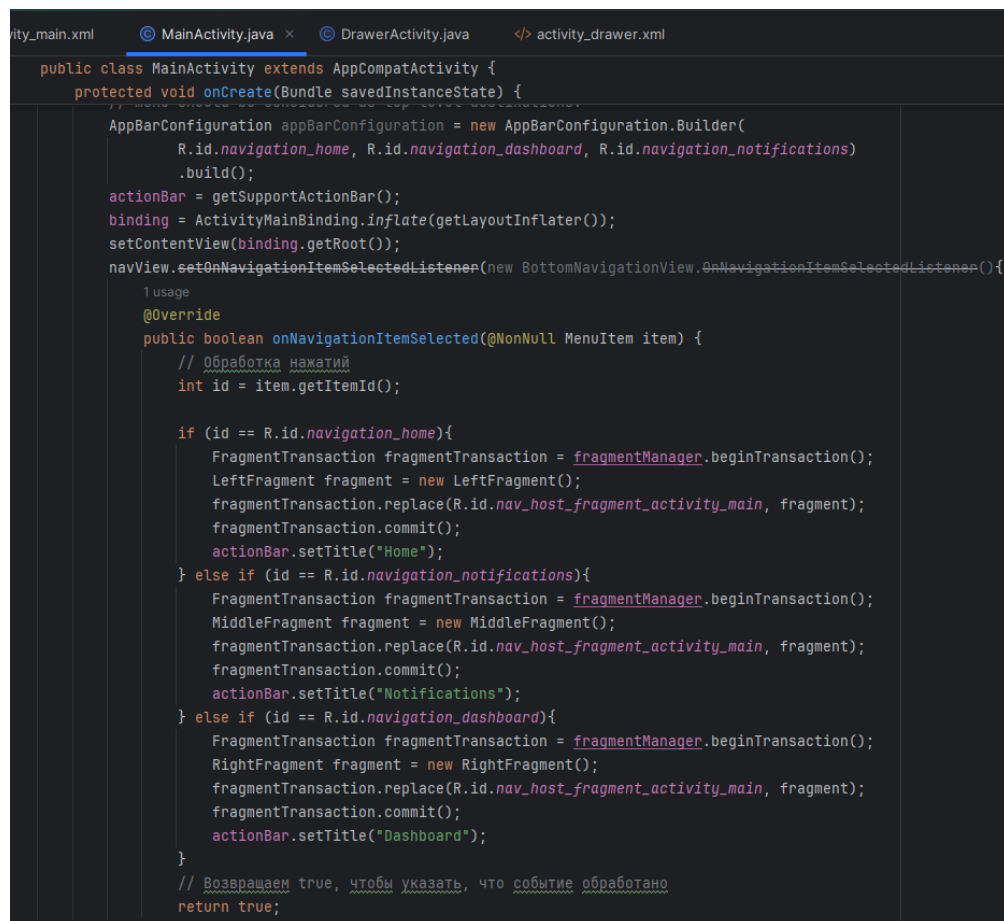


Рисунок 2.2.2 – Реализация логики нижнего меню

При этом на рисунке 2.2.2 отображена реализация обработки событий, происходящих в нижнем меню.

В итоге приложение будет отображать различные фрагменты в зависимости от выбора пользователем элемента из нижнего меню (рисунки 2.2.3-2.2.5).

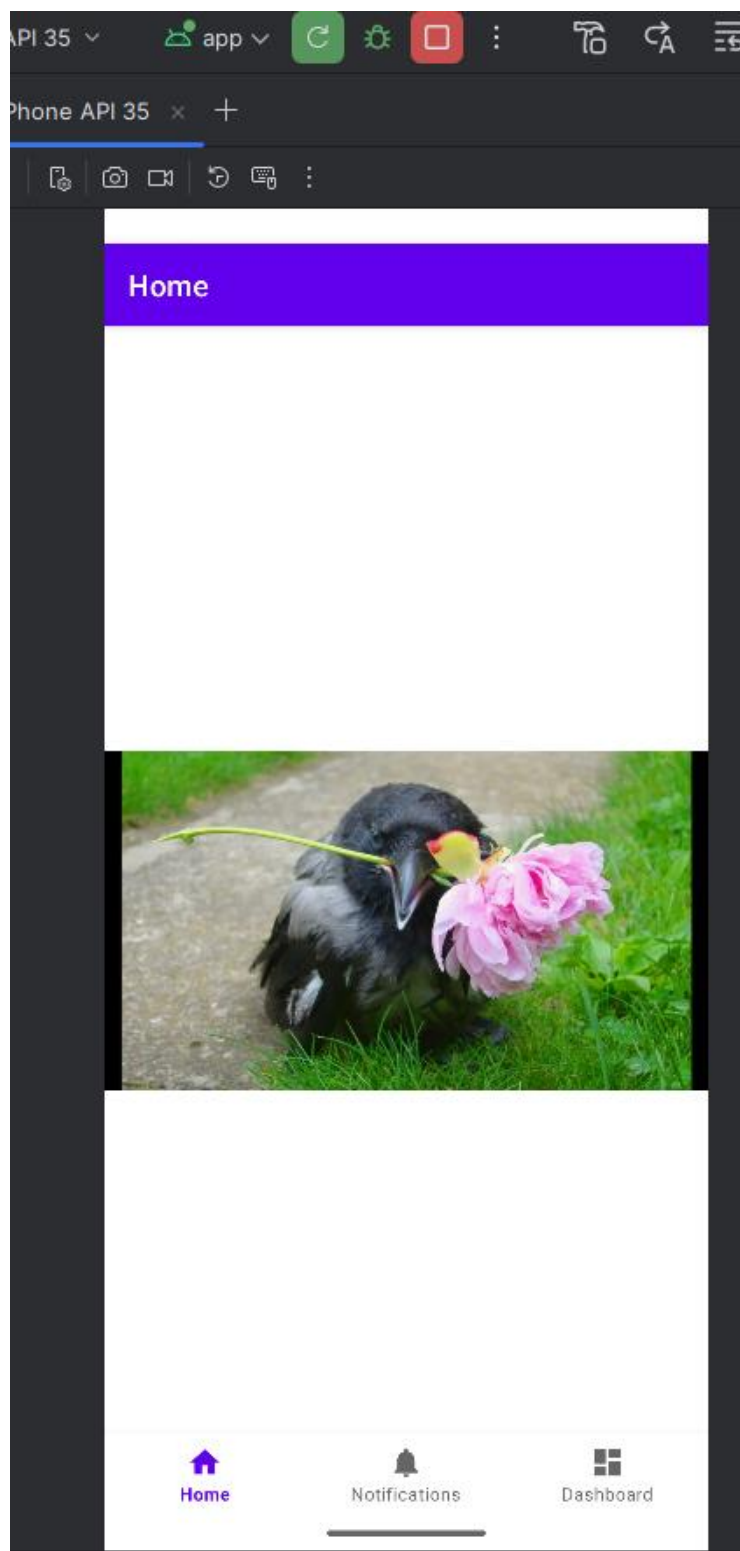


Рисунок 2.2.3 – Результат нажатия пользователем на элемент Home

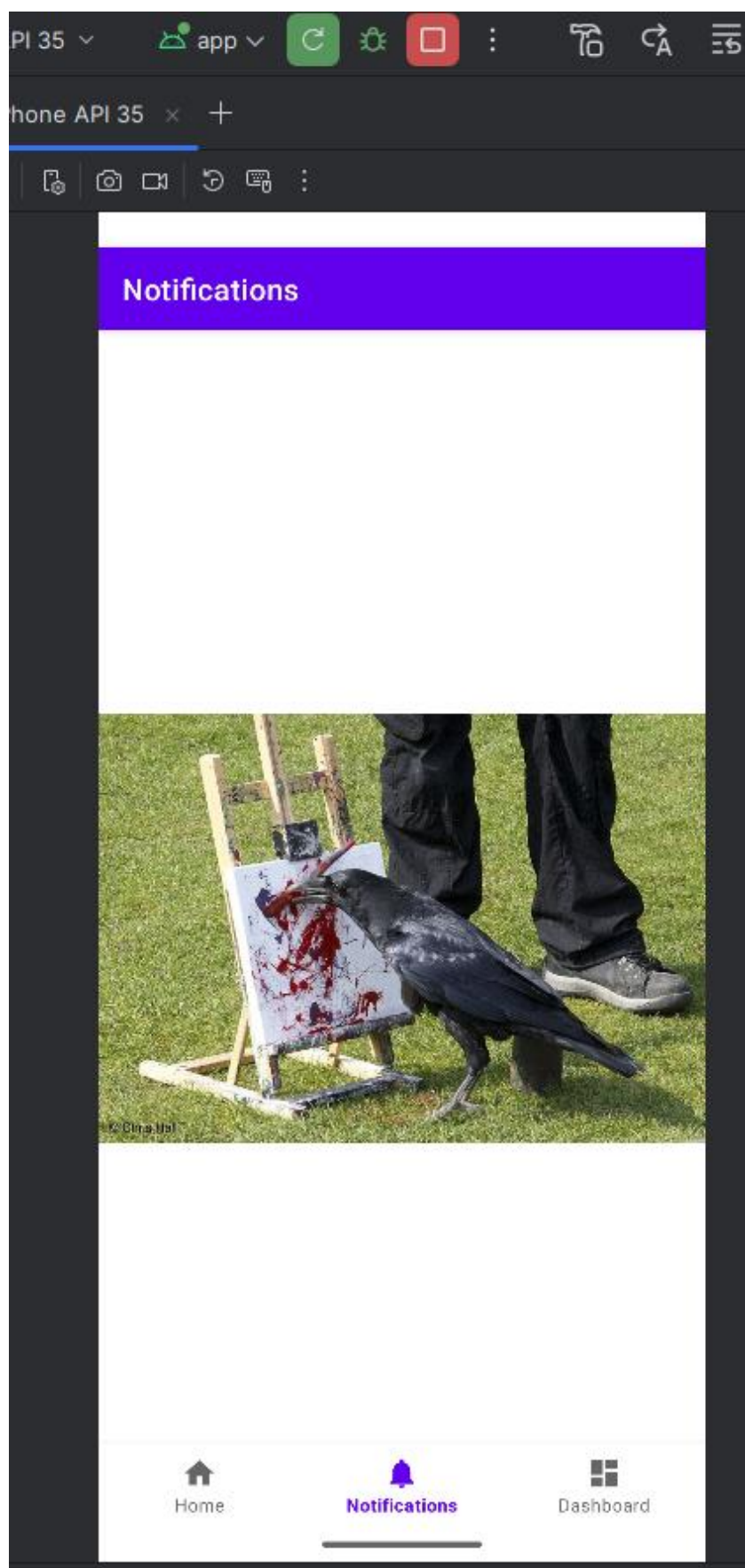


Рисунок 2.2.4 – Результат нажатия пользователем на элемент Notifications

Как и на рисунке 2.2.3, на рисунке 2.2.4 отображено перемещению между фрагментами и замена заголовка ActionBar-a.

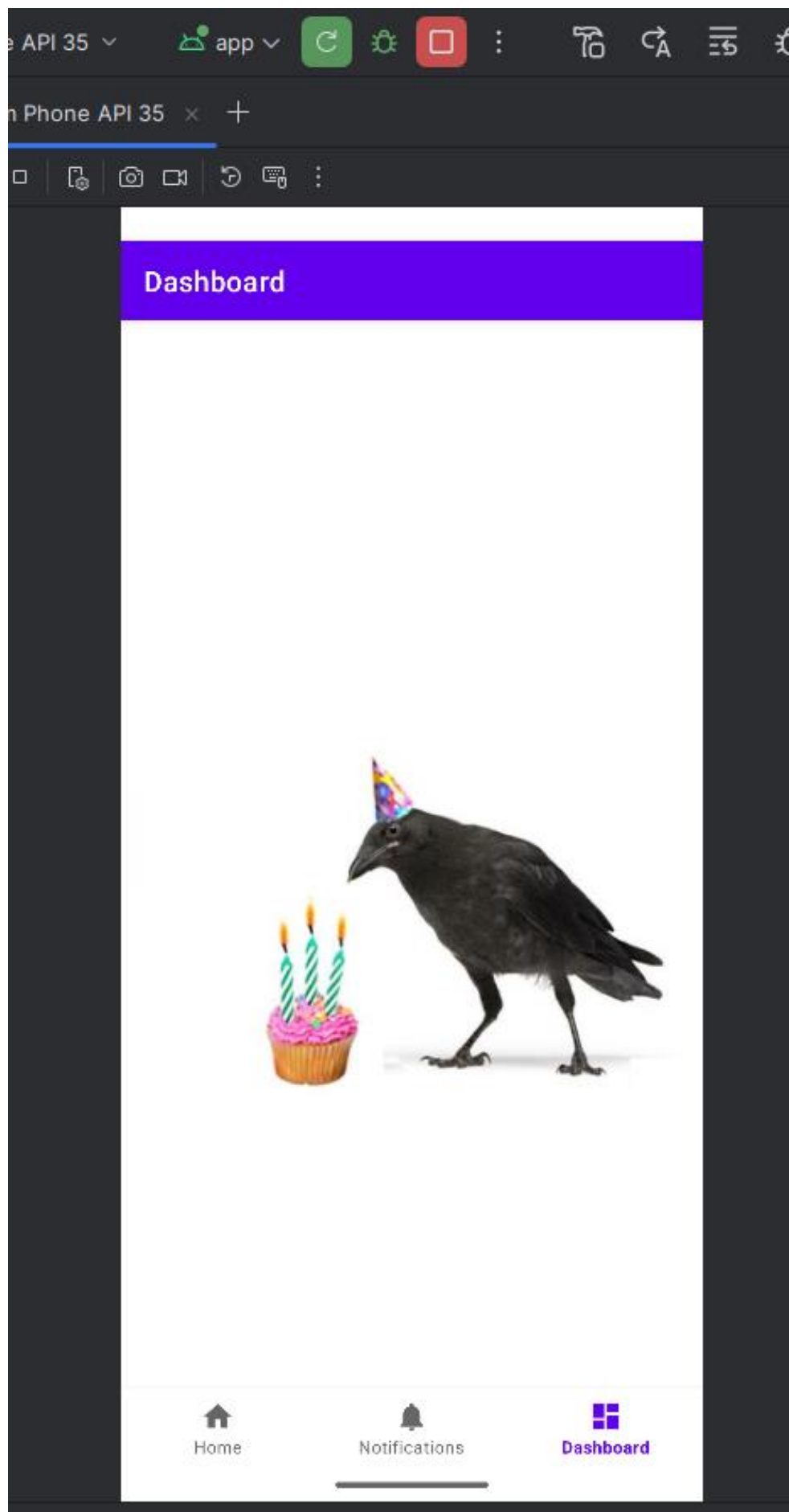


Рисунок 2.2.5 – Результат нажатия пользователем на элемент Dashboard

ЗАКЛЮЧЕНИЕ

В ходе данной практической работы были получены знания связанные с `NavigationBar`, `ActionBar` и `BottomBar`. Полученные знания были закреплены путём выполнения практического задания.